

Ecolingo — Guía de Código

Documento único de referencia técnica para construir la plataforma, de cero a versión final.

Campo	Valor
Proyecto	Ecolingo — Plataforma Web Gamificada para Educación Financiera
Institución	Universidad Columbia del Paraguay — Ingeniería Informática
Autores	Cesar Luis Alcaraz Casco · Martín Vega
Tutor	Prof. José Alfredo Souberlich Román
Versión del documento	2.0
Fecha	Agosto 2026
Reemplaza a	ECOLINGO_CONTRATO_TECNICO.md v1.1, ECOLINGO_GUIA_BASE_DE_DATOS.md v1.1, 02-FEATURES.md v1.0, 01-PRE-CODIGO.md, y las referencias funcionales del Documento de Diseño de Interfaces y del Brief de Cesar
Estado	Vigente — requiere acuerdo bilateral para modificarse

Cómo se usa este documento

Este archivo reemplaza a los nueve documentos anteriores. Tiene cuatro partes, en el orden en que se construye la plataforma:

1. **Antes de escribir código** — todo lo que hay que tener definido antes de la primera línea: paleta, convenciones de API, JWT, variables de entorno, esqueleto del backend, reglas de negocio, y el mapa de las 12 features con el orden de trabajo.
2. **API por feature** — los endpoints de cada feature, en el orden en que se construyen: servidor primero.
3. **Base de datos por feature** — los modelos de Mongoose y seeds que necesita cada feature.
4. **Frontend por feature** — la pantalla, las rutas, los colores y la funcionalidad de cada feature.

Regla de uso en el chat de una feature: al abrir un chat nuevo para trabajar una feature, se sube este documento y se dice qué feature es y quién sos. Ejemplo:

En este chat trabajamos la feature **F04 — Motor de Lección**. Soy la Persona B. Usá la Parte 1 (API), Parte 2 (BD) y Parte 3 (Frontend) de esa feature en [GUIA_DE_CODIGO.md](#).

Regla de gobernanza. Ninguno de los dos modifica unilateralmente lo definido acá. Cualquier cambio requiere acuerdo explícito entre ambos y un commit `docs(guia): <descripción>` en `main`.

Decisiones ya cerradas en esta versión (no vuelvas a discutir las salvo que aparezca información nueva):

- Los alumnos se autentican con **PIN de 4 dígitos** (reemplaza la política "sin contraseña" de versiones anteriores). Ver Parte 0.2 y D-05 revisada.
- **F11 (contenido teórico) se implementa.** Cada módulo tiene un bloque de teoría antes de las preguntas. Ver Parte 1 — F11 y D-16.
- Persona A / Persona B: **todavía sin asignar.** Es el primer punto del checklist de arranque (Parte 0.10).

PARTE 0 — Antes de escribir código

Todo lo que hay que tener definido antes de la primera línea de código. Se revisa una vez, en pareja, y no se retoca sin acuerdo bilateral.

0.1 Convenciones generales, formatos, códigos HTTP y catálogo de errores

Convenciones de nomenclatura

Ámbito	Convención	Ejemplo
Campos JSON (request y response)	<code>camelCase</code>	<code>totalXp, classCode</code>
Campos de MongoDB	<code>camelCase</code>	<code>passwordHash</code>
Nombres de colecciones	plural, <code>camelCase</code>	<code>users, moduleAttempts</code>
Rutas de API	<code>kebab-case</code> , sustantivos en plural	<code>/api/v1/module-attempts</code>
Parámetros de ruta	<code>camelCase</code> con prefijo <code>:</code>	<code>:moduleId</code>
Códigos de error	<code>SCREAMING_SNAKE_CASE</code>	<code>INVALID_CLASS_CODE</code>
Variables de entorno	<code>SCREAMING_SNAKE_CASE</code>	<code>MONGODB_URI</code>
Ramas de Git	<code><tipo>/<feature></code>	<code>feat/mapa-modulos</code>

Formatos de datos

Tipo	Formato	Ejemplo
Identificadores	<code>ObjectId</code> de MongoDB, serializado como string hexadecimal de 24 caracteres	<code>"66b1f0a2c3d4e5f6a7b8c9d0"</code>
Fechas y horas	ISO 8601 en UTC, con sufijo <code>Z</code>	<code>"2026-09-15T14:32:00.000Z"</code>
Fechas sin hora	ISO 8601 solo fecha	<code>"2026-09-15"</code>
Duraciones	Enteros en segundos	<code>172</code>
Booleanos	<code>true</code> / <code>false</code> (nunca <code>1/0</code> , nunca <code>"true"</code>)	<code>true</code>

Tipo	Formato	Ejemplo
Valores ausentes	<code>null</code> explícito (nunca omitir la clave en una respuesta documentada)	<code>"completedAt": null</code>

Regla crítica: el campo `_id` de MongoDB **nunca** se expone tal cual en la API. Toda respuesta lo serializa como `id` (string). Esto se implementa una sola vez con un transform global en Mongoose:

```
// backend/src/config/mongoose.js
mongoose.set('toJSON', {
  virtuals: true,
  versionKey: false,
  transform: (doc, ret) => {
    ret.id = ret._id.toString();
    delete ret._id;
    delete ret.passwordHash;
    return ret;
  },
});
```

Versionado y URL base

Todas las rutas se montan bajo el prefijo `/api/v1`. En el frontend se consume mediante una única instancia de Axios:

```
// frontend/src/api/client.js
const client = axios.create({
  baseURL: import.meta.env.VITE_API_URL, // ej: https://ecolingo-
  api.onrender.com/api/v1
  timeout: 10000,
});
```

Entorno	VITE_API_URL
Desarrollo local	<code>http://localhost:4000/api/v1</code>
Producción (Render)	<code>https://<subdominio>.onrender.com/api/v1</code>

Formato de respuesta

Éxito. El recurso se devuelve directamente en el cuerpo, sin envoltorio. El código HTTP comunica el resultado.

```
{ "id": "66b1...", "fullName": "Ana Rodríguez", "totalXp": 350 }
```

Las colecciones se devuelven siempre como objeto con la clave del recurso, nunca como array desnudo (permite agregar metadatos como paginación sin romper el contrato):

```
{ "modules": [ ... ], "total": 6 }
```

Error. Siempre el mismo envoltorio, en todos los endpoints, sin excepción:

```
{
  "error": {
    "code": "INVALID_CLASS_CODE",
    "message": "El código de clase no existe o no está activo.",
    "details": []
  }
}
```

Campo	Tipo	Descripción
<code>error.code</code>	string	Identificador estable, legible por máquina. El frontend hace <code>switch</code> sobre este valor.
<code>error.message</code>	string	Mensaje en español, apto para mostrar directamente al usuario final.
<code>error.details</code>	array	Errores de validación campo por campo. Vacío si no aplica.

Estructura de cada elemento de `details`:

```
{ "field": "classCode", "issue": "Debe tener entre 4 y 12 caracteres." }
```

Códigos HTTP en uso

Código	Uso en Ecolingo
<code>200 OK</code>	Lectura exitosa o mutación que no crea un recurso.
<code>201 Created</code>	Recurso creado (registro de alumno, inicio de intento).
<code>204 No Content</code>	Eliminación o acción sin cuerpo de respuesta.
<code>400 Bad Request</code>	Cuerpo malformado o validación fallida.
<code>401 Unauthorized</code>	Token ausente, inválido o expirado.
<code>403 Forbidden</code>	Token válido pero rol o propiedad insuficiente.
<code>404 Not Found</code>	Recurso inexistente.
<code>409 Conflict</code>	Violación de una regla de negocio o unicidad.

Código	Uso en Ecolingo
422 Unprocessable Entity	Petición sintácticamente válida pero semánticamente imposible (ej. responder un intento ya finalizado).
500 Internal Server Error	Error no controlado. Nunca se expone el stack trace.

Catálogo completo de códigos de error

code	HTTP	Cuándo se emite
VALIDATION_ERROR	400	Uno o más campos del body no cumplen las reglas. details poblado.
MALFORMED_ID	400	Un parámetro de ruta no es un ObjectId válido.
MISSING_TOKEN	401	No se envió el header Authorization .
INVALID_TOKEN	401	Firma inválida o token corrupto.
EXPIRED_TOKEN	401	El token superó su tiempo de vida.
INVALID_CREDENTIALS	401	Email o contraseña de docente incorrectos.
FORBIDDEN_ROLE	403	El rol del token no habilita el endpoint.
NOT_RESOURCE_OWNER	403	El usuario intenta acceder a un recurso de otro usuario.
USER_NOT_FOUND	404	El usuario referenciado no existe.
MODULE_NOT_FOUND	404	El módulo referenciado no existe.
ATTEMPT_NOT_FOUND	404	El intento referenciado no existe.
CLASS_NOT_FOUND	404	El código de clase no existe.
INVALID_CLASS_CODE	404	El código de clase no existe o está inactivo (usado en login de alumno).
EMAIL_ALREADY_REGISTERED	409	Ya existe un docente con ese email.
STUDENT_CODE_TAKEN	409	Ya existe un alumno con ese studentCode en la clase.
MODULE_LOCKED	409	Se intenta iniciar un módulo cuyo prerrequisito no está completo.
ATTEMPT_ALREADY_ACTIVE	409	Ya hay un intento en curso para ese módulo.
ATTEMPT_ALREADY_FINISHED	422	Se intenta responder o abandonar un intento ya cerrado.
QUESTION_ALREADY_ANSWERED	422	Se reenvía una respuesta a una pregunta ya respondida en ese intento.
TEST_NOT_ENABLED	403	Se consulta el pre/post-test sin que el docente lo haya habilitado.

code	HTTP	Cuándo se emite
INVALID_INTEGRATION_KEY	401	La clave del webhook de Google Forms es incorrecta.
INTERNAL_ERROR	500	Error no controlado.
INCORRECT_CURRENT_PASSWORD	401	La contraseña actual enviada al cambiar credenciales no coincide.
SEARCH_QUERY_TOO_SHORT	400	El parámetro q del buscador tiene menos de 2 caracteres.

0.2 JWT y modelo de autorización

Estructura del token

Algoritmo: **HS256**. Firmado con **JWT_SECRET**. Vida útil: **7 días** (**JWT_EXPIRES_IN=7d**), suficiente para las 4 semanas de piloto sin forzar re-logins frecuentes durante una sesión de clase.

Payload exacto — idéntico para ambos roles:

```
{
  "sub": "66b1f0a2c3d4e5f6a7b8c9d0",
  "role": "student",
  "fullName": "Ana Rodríguez",
  "classCode": "EC02025",
  "iss": "ecolingo-api",
  "aud": "ecolingo-web",
  "iat": 1755691200,
  "exp": 1756296000
}
```

Claim	Tipo	Obligatorio	Descripción
sub	string	Sí	id del documento en users . Claim estándar de sujeto.
role	string	Sí	"student" "teacher". Determina el enrutamiento y los permisos.
fullName	string	Sí	Nombre para mostrar en la UI sin pedir un GET /me adicional.
classCode	string null	Sí	Código de clase del alumno. null para docentes (pueden tener varias clases).
iss	string	Sí	Emisor. Valor fijo "ecolingo-api".
aud	string	Sí	Audiencia. Valor fijo "ecolingo-web".
iat	number	Sí	Emitido en (epoch en segundos). Lo agrega jsonwebtoken .

Claim	Tipo	Obligatorio	Descripción
<code>exp</code>	number	Sí	Expira en (epoch en segundos). Lo agrega <code>jsonwebtoken</code> .

Qué NO va en el token, y por qué:

Campo	Motivo de exclusión
<code>totalXp, streak</code>	Cambian constantemente. Un token no puede quedar desactualizado respecto al estado del juego.
<code>email</code>	No aporta a la autorización y expone un dato personal en un valor que se guarda en el navegador.
<code>cedula / studentCode</code>	Dato sensible. Nunca debe viajar en un token almacenado en <code>localStorage</code> .
<code>passwordHash</code>	Nunca, bajo ninguna circunstancia.

Firma y verificación

```
// backend/src/utils/jwt.js
import jwt from 'jsonwebtoken';

export const signToken = (user) =>
  jwt.sign(
    {
      sub: user.id,
      role: user.role,
      fullName: user.fullName,
      classCode: user.classCode ?? null,
    },
    process.env.JWT_SECRET,
    {
      algorithm: 'HS256',
      expiresIn: process.env.JWT_EXPIRES_IN,
      issuer: 'ecolingo-api',
      audience: 'ecolingo-web',
    }
  );

export const verifyToken = (token) =>
  jwt.verify(token, process.env.JWT_SECRET, {
    algorithms: ['HS256'],
    issuer: 'ecolingo-api',
    audience: 'ecolingo-web',
  });
```

Middleware de autorización

Ambos desarrolladores usan estos tres middlewares. Se implementan una sola vez, en la fase común de autenticación.

Middleware	Comportamiento	Error si falla
<code>requireAuth</code>	Verifica el token y adjunta <code>req.user</code> = { <code>id</code> , <code>role</code> , <code>fullName</code> , <code>classCode</code> }.	<code>MISSING_TOKEN</code> / <code>INVALID_TOKEN</code> / <code>EXPIRED_TOKEN</code>
<code>requireRole('student')</code>	Exige <code>req.user.role === 'student'</code> . Se aplica después de <code>requireAuth</code> .	<code>FORBIDDEN_ROLE</code>
<code>requireRole('teacher')</code>	Exige <code>req.user.role === 'teacher'</code> .	<code>FORBIDDEN_ROLE</code>

Regla de propiedad de recursos. Ningún endpoint de alumno acepta un `userId` por parámetro o body. La identidad siempre se deriva de `req.user.id`. Esto elimina por diseño toda una clase de vulnerabilidad de acceso horizontal (IDOR) y es un punto defendible ante el tribunal.

0.2.1 Autenticación de alumnos con PIN

Los alumnos se autentican con `fullName` + `studentCode` + `classCode` + un **PIN numérico de 4 dígitos**. Esto reemplaza la política anterior ("sin contraseña"): con solo nombre y código, cualquiera que conociera esos dos datos de un compañero —plausible en un curso de secundaria, donde los códigos suelen ser correlativos— podía autenticarse como él. El PIN agrega un secreto mínimo, apropiado para la edad de los usuarios y sin la fricción de una contraseña completa.

Cómo se establece. El alumno **elige su propio PIN en el primer ingreso** — no hay un paso previo de configuración ni un PIN generado por el docente. Es el mismo endpoint de siempre (`POST /auth/student/login`), con un campo más en el body.

Cómo se recupera. Si el alumno lo olvida, **no hay autorecuperación**. Avisa al docente, que le genera un PIN nuevo desde el panel (endpoint 26, Parte 1 — F01) y se lo comunica verbalmente en el aula. Nunca por escrito en un canal inseguro — son datos de menores.

Almacenamiento. `pinHash` en `users`, con `bcrypt` y el mismo `BCRYPT_SALT_ROUNDS` que la contraseña del docente. Nunca se guarda ni se expone en texto plano, salvo en la respuesta única e inmediata del endpoint de reseteo.

Validación. `^\d{4}$` — exactamente 4 dígitos. Se transmite como string, no como número, para no perder ceros a la izquierda ("`0092`" es un PIN válido).

0.3 Autenticación en las peticiones (interceptor de Axios)

Todo endpoint protegido exige:

```
Authorization: Bearer <jwt>
```


El interceptor de Axios lo inyecta automáticamente y centraliza el manejo de expiración:

```
client.interceptors.request.use((config) => {
  const token = localStorage.getItem('ecolingo_token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

client.interceptors.response.use(
  (res) => res,
  (err) => {
    const code = err.response?.data?.error?.code;
    if (code === 'EXPIRED_TOKEN' || code === 'INVALID_TOKEN') {
      localStorage.removeItem('ecolingo_token');
      window.location.assign('/login');
    }
    return Promise.reject(err);
  }
);
```

0.4 Paleta de colores (tokens de Tailwind)

Se declaran una sola vez, en `tailwind.config.js`, a partir de la paleta del Documento de Diseño de Interfaces. **Ningún componente usa colores hexadecimales literales.**

```
// tailwind.config.js
export default {
  content: ['./index.html', './src/**/*.{js,jsx}'],
  theme: {
    extend: {
      colors: {
        bg: '#0d1117', // fondo de la aplicación
        surface: '#161b22', // cards, módulos, filas de ranking
        primary: '#1a6b3a', // botones principales, bordes activos
        accent: '#4ade80', // XP, destacados, respuestas correctas
        info: '#60a5fa', // módulo activo, posición propia en ranking
        warn: '#f59e0b', // racha diaria, estadísticas de posición
        danger: '#ef4444', // respuestas incorrectas, vida perdida
        text: '#e6f5ee', // títulos y texto principal
        muted: '#8b949e', // descripciones, subtítulos, etiquetas
      },
      borderWidth: { hair: '0.5px' },
    },
  },
  plugins: [],
};
```

0.5 Variables de entorno

Backend (**backend/.env**)

Variable	Ejemplo	Descripción
PORT	4000	Puerto del servidor Express.
NODE_ENV	development	development production.
MONGODB_URI	mongodb+srv://...	Cadena de conexión a MongoDB Atlas.
JWT_SECRET	(cadena aleatoria de 64 caracteres)	Clave de firma HS256.
JWT_EXPIRES_IN	7d	Vida del token.
BCRYPT_SALT_ROUNDS	10	Coste del hash de contraseña.
CORS_ORIGIN	http://localhost:5173	Origen permitido (coma-separado en producción).
FORMS_INTEGRATION_KEY	(cadena aleatoria)	Clave compartida con Google Apps Script (Fase 2).

Frontend (**frontend/.env**)

Variable	Ejemplo
VITE_API_URL	http://localhost:4000/api/v1

El archivo **.env** nunca se commitea. Ambos mantienen un **.env.example** actualizado con las claves pero sin los valores.

0.6 Estructura de carpetas

```

ecolingo/
├── backend/
│   ├── src/
│   │   ├── config/      # conexión a Mongo, CORS, variables de entorno
│   │   ├── models/      # ⚠ COMPARTIDO – cambios requieren acuerdo
│   │   ├── middlewares/  # ⚠ COMPARTIDO – requireAuth, requireRole,
│   │   ├── errorHandler
│   │   ├── utils/       # ⚠ COMPARTIDO – jwt.js, date.js, apiError.js
│   │   ├── services/    # xpService, badgeService, streakService,
│   │   ├── rankingService
│   │   ├── controllers/
│   │   ├── routes/
│   │   ├── seeds/       # modules.js, questions.js, badges.js, teacher.js
│   │   └── app.js

```

```

├── server.js
├── .env.example
└── frontend/
    └── src/
        ├── api/           # ⚠ COMPARTIDO – client.js con interceptores
        ├── context/       # ⚠ COMPARTIDO – AuthContext
        ├── components/
        │   └── common/    # ⚠ COMPARTIDO – Button, Card, ProgressBar,
Spinner
        │   └── <feature>/ # territorio de cada desarrollador
        ├── pages/
        ├── hooks/
        ├── styles/       # ⚠ COMPARTIDO – tokens de Tailwind
        └── App.jsx        # ⚠ COMPARTIDO – definición de rutas
    
```

Regla de zonas compartidas. Los directorios marcados con ⚠ son de propiedad conjunta. Antes de modificar cualquier archivo dentro de ellos, se avisa al otro desarrollador. Los directorios de feature son de propiedad exclusiva: dentro de ellos cada uno decide sin consultar.

0.7 Conexión a MongoDB y esqueleto de Express

src/config/database.js

```

import mongoose from 'mongoose';

// Transform global: toda respuesta expone `id` (string) en lugar de `_id`,
// y nunca serializa `passwordHash`. Definido en la §0.2 del Contrato
Técnico.
mongoose.set('toJSON', {
  virtuals: true,
  versionKey: false,
  transform: (doc, ret) => {
    ret.id = ret._id?.toString();
    delete ret._id;
    delete ret.passwordHash;
    return ret;
  },
});

// En desarrollo, imprime cada consulta en la consola. Muy útil para
aprender
// qué está haciendo Mongoose por debajo.
if (process.env.NODE_ENV === 'development') {
  mongoose.set('debug', true);
}

export const connectDatabase = async () => {
  const uri = process.env.MONGODB_URI;
  if (!uri) throw new Error('Falta la variable de entorno MONGODB_URI');
}
    
```

```

    try {
      const conn = await mongoose.connect(uri, {
        serverSelectionTimeoutMS: 10000,
        autoIndex: process.env.NODE_ENV !== 'production',
      });
      console.log(`✅ MongoDB conectado — base: ${conn.connection.name}`);
      return conn;
    } catch (error) {
      console.error(`❌ Error al conectar con MongoDB:`, error.message);
      process.exit(1);
    }
  };

  mongoose.connection.on('disconnected', () => console.warn('⚠️ MongoDB desconectado'));
  mongoose.connection.on('reconnected', () => console.log('✅ MongoDB reconectado'));

```

Sobre `autoIndex`. Con `autoIndex: true`, Mongoose crea los índices declarados cada vez que arranca la aplicación. Es cómodo en desarrollo. En producción se desactiva porque crear índices bloquea la colección: si el servidor se reinicia mientras un alumno está jugando, se produce una pausa. Los índices de producción se crean una sola vez, con el script de la §4.4.

Aplicación Express mínima

src/app.js

```

import express from 'express';
import cors from 'cors';
import mongoose from 'mongoose';

const app = express();

app.use(cors({ origin: process.env.CORS_ORIGIN?.split(',') ?? '*' }));
app.use(express.json({ limit: '100kb' }));

// Endpoint 1 del Contrato Técnico
app.get('/api/v1/health', (req, res) => {
  const states = ['disconnected', 'connected', 'connecting', 'disconnecting'];
  res.json({
    status: 'ok',
    environment: process.env.NODE_ENV,
    database: states[mongoose.connection.readyState] ?? 'unknown',
    timestamp: new Date().toISOString(),
  });
});

// 404 con el envoltorio de error estándar (§0.4 del Contrato Técnico)
app.use((req, res) => {
  res.status(404).json({

```

```
      error: { code: 'NOT_FOUND', message: 'Recurso no encontrado.', details:
    [] },
    });
  });

export default app;
```

src/server.js

```
import 'dotenv/config';
import app from './app.js';
import { connectDatabase } from './config/database.js';

const PORT = process.env.PORT || 4000;

await connectDatabase();

app.listen(PORT, () => {
  console.log(`🚀 Servidor escuchando en http://localhost:${PORT}`);
});
```

Primera prueba

```
npm run dev
```

Salida esperada:

```
✅ MongoDB conectado – base: ecolingo_dev
🚀 Servidor escuchando en http://localhost:4000
```

Verificá el endpoint:

```
curl http://localhost:4000/api/v1/health
```

```
{"status":"ok","environment":"development","database":"connected","timestamp":"2026-08-08T..."}
```

Si **database** dice **"connecting"** o **"disconnected"**, andá al Bloque 10.

0.8 Reglas de negocio transversales (XP, vidas, racha, insignias, huso horario)

Estas reglas se implementan en el backend, en servicios compartidos, y **ninguna** se replica en el frontend. El cliente solo renderiza lo que el servidor decide.

Cálculo de XP

Regla	Definición
XP por respuesta correcta	50, constante en todos los módulos.
XP por respuesta incorrecta	0.
Acreditación	Solo en la primera finalización del módulo (<code>progress.xpEarned === 0</code>).
Intentos posteriores	Actualizan estadísticas, otorgan 0 XP.
Intentos <code>failed</code> o <code>abandoned</code>	Otorgan 0 XP.
Escritura	Cada acreditación incrementa <code>users.totalXp</code> y <code>users.weeklyXp</code> de forma atómica (<code>\$inc</code>).

Sistema de vidas

Regla	Definición
Vidas iniciales	3 por intento.
Pérdida	-1 por respuesta incorrecta.
Agotamiento	Con 0 vidas el intento pasa a <code>failed</code> de inmediato.
Reposición	Ninguna dentro del intento. Un intento nuevo reinicia en 3.
Límite de reintentos	Sin límite. La repetición es deseable pedagógicamente y no distorsiona el ranking gracias a la regla anti-farming.

Cálculo de racha

Se evalúa al finalizar cualquier intento, comparando `lastActivityDate` con la fecha actual en **UTC-3**:

Condición	Efecto
<code>lastActivityDate === hoy</code>	Sin cambios. Ya contó hoy.
<code>lastActivityDate === ayer</code>	<code>currentStreak += 1</code> .
<code>lastActivityDate</code> anterior a ayer, o <code>null</code>	<code>currentStreak = 1</code> .

Condición	Efecto
Siempre	<code>longestStreak = max(longestStreak, currentStreak);</code> <code>lastActivityDate = hoy.</code>

La racha se calcula sobre **actividad real** (finalizar un intento), no sobre el mero login. Un alumno que abre la aplicación sin jugar no acumula racha. Esta es una decisión pedagógica que hace de la métrica un indicador de engagement válido para el Capítulo 5.

Evaluación de insignias

Se ejecuta en el servicio `evaluateBadges(userId, attemptId)`, invocado al finalizar un intento. Es idempotente: el índice único de `userBadges` impide duplicados.

Insignia	Condición evaluada
<code>PRIMER_AHORRO ...</code> <code>PLANIFICADOR</code>	<code>progress.status === "completed"</code> para el módulo del <code>criteriaValue</code> .
<code>MODULOS_COMPLETOS</code>	Los 6 módulos con <code>status: "completed"</code> .
<code>RESPUESTA_PERFECTA</code>	<code>attempt.correctCount === module.questionCount</code> .
<code>VELOCISTA</code>	<code>attempt.status === "completed"</code> y <code>durationSeconds < 180</code> .
<code>RACHA_7_DIAS</code>	<code>currentStreak >= 7</code> , evaluada tras actualizar la racha.
<code>TOP_3_PODIO</code>	Tras acreditar XP, el alumno queda en posición ≤ 3 del ranking semanal de su clase.

Zona horaria

Todas las fechas se **almacenan en UTC**. Todos los cálculos con semántica de día calendario — racha, `activeToday`, corte semanal del ranking — se realizan en **UTC-3 (America/Asuncion)**. La conversión ocurre en una única utilidad del backend:

```
// backend/src/utils/date.js – única fuente de verdad para fechas locales
const PY_OFFSET_HOURS = -3;
export const toLocalDateString = (date = new Date()) => { /* → 'YYYY-MM-DD' */ };
export const getWeekStart = (date = new Date()) => { /* → lunes 00:00 UTC-3 en UTC */ };
```

Paraguay dejó de aplicar horario de verano a partir de 2024, por lo que el desfase de -3 es constante durante todo el período del piloto. Este supuesto queda documentado explícitamente en el código.

0.9 Diagrama de relaciones de datos

```

classes —1:N→ users (students)
  |
  | teacherId
  |
users (teacher)
  |
  | —1:N→ progress —N:1→ modules —1:N→ questions
  |
  | —1:N→ moduleAttempts —N:1→ modules
  |
  | —1:N→ userBadges —N:1→ badges
  |
  | —1:1→ testResults (Fase 2, vía studentCode)
  
```

0.10 Índices — inventario completo

Colección	Índice	Tipo	Para qué
classes	{ code: 1 }	único	Login de alumno: buscar la clase por código.
classes	{ teacherId: 1 }	simple	Listar las clases de un docente.
users	{ email: 1 }	único + disperso	Login de docente. Disperso porque los alumnos tienen email: null .
users	{ classCode: 1, studentCode: 1 }	único + disperso	Regla de negocio: código único por clase.
users	{ classCode: 1, weeklyXp: -1 }	compuesto	Ranking semanal: filtra y ordena con un solo índice.
users	{ classCode: 1, totalXp: -1 }	compuesto	Panel del docente y ranking histórico.
modules	{ order: 1 }	único	Orden del mapa y lógica de desbloqueo.
modules	{ slug: 1 }	único	Acceso por identificador legible.
questions	{ moduleId: 1, order: 1 }	único	Servir la pregunta N del módulo M.
progress	{ userId: 1, moduleId: 1 }	único	Regla de negocio: un documento por par.
progress	{ userId: 1, status: 1 }	compuesto	Contar módulos completados.
moduleAttempts	{ userId: 1, moduleId: 1, status: 1 }	compuesto	Buscar el intento activo.

Colección	Índice	Tipo	Para qué
<code>moduleAttempts</code>	<code>{ userId: 1, moduleId: 1 }</code>	único parcial	Regla de negocio: un solo intento activo.
<code>badges</code>	<code>{ code: 1 }</code>	único	Referencia desde <code>modules</code> y <code>userBadges</code> .
<code>userBadges</code>	<code>{ userId: 1, badgeCode: 1 }</code>	único	Regla de negocio: idempotencia.
<code>testResults</code>	<code>{ studentCode: 1, testType: 1 }</code>	único	Regla de negocio: un resultado por alumno y test.
<code>glossaryTerms</code>	<code>{ slug: 1 }</code>	único	Acceso al término por su identificador de URL.
<code>glossaryTerms</code>	<code>{ term, definition, keywords }</code>	texto	Buscador (F10). Español, con pesos por campo. Único índice de texto de la base.

`src/scripts/createIndexes.js`

```
import 'dotenv/config';
import mongoose from 'mongoose';
import { connectDatabase } from '../config/database.js';
import * as models from '../models/index.js';

await connectDatabase();

for (const [name, model] of Object.entries(models)) {
  await model.createIndexes();
  const idx = await model.collection.indexes();
  console.log(`✅ ${name}: ${idx.length} índices`);
}

await mongoose.disconnect();
console.log('🏁 Índices creados.');
```

```
node src/scripts/createIndexes.js
```

0.11 Orquestador de seeds

`src/seeds/index.js`

```
import 'dotenv/config';
import mongoose from 'mongoose';
```

```
import { connectDatabase } from '../config/database.js';
import { seedBadges } from './badges.js';
import { seedModules } from './modules.js';
import { seedQuestions } from './questions.js';
import { seedClassroom } from './classroom.js';
import { seedGlossary } from './glossary.js';

const run = async () => {
  await connectDatabase();
  const dbName = mongoose.connection.name;

  console.log(`\n🌱 Seed sobre la base: ${dbName}\n`);

  // Salvaguarda: exige confirmación explícita para tocar producción.
  if (dbName.includes('prod') && process.env.CONFIRM_PROD_SEED !== 'yes') {
    console.error(
      '⚠️ Estás apuntando a la base de PRODUCCIÓN.\n' +
      'Si es intencional, ejecutá: CONFIRM_PROD_SEED=yes npm run seed'
    );
    process.exit(1);
  }

  // El orden importa: módulos antes que preguntas, docente antes que
  // clase.
  await seedBadges();
  await seedModules();
  await seedQuestions();
  await seedClassroom();
  await seedGlossary();

  console.log(`\n🏁 Seed completado.\n`);
  await mongoose.disconnect();
};

run().catch(async (err) => {
  console.error('❌ El seed falló:', err.message);
  await mongoose.disconnect();
  process.exit(1);
});
```

Esa salvaguarda vale su peso en oro. El escenario que previene es concreto: es octubre, estás cansado, tenés la terminal apuntando a producción sin darte cuenta, corrés `npm run seed` y sobrescribís datos del piloto en curso. Nueve líneas de código que protegen el trabajo de cuatro meses.

0.12 Rutas del frontend

Definidas en `App.jsx`, zona compartida.

Ruta	Pantalla	Guard	Responsable
/login	Login	público	Ambos
/mapa	Mapa de Módulos	student	Persona A
/leccion/:moduleId/teoria	Contenido teórico (F11)	student	Persona B
/leccion/:attemptId	Lección (motor de preguntas)	student	Persona B
/resultado/:attemptId	Resultado	student	Ambos
/ranking	Ranking de Clase	student	Persona A
/logros	Logros e Insignias	student	Persona A
/perfil	Perfil de usuario (F09)	student teacher	Persona A
/buscar	Buscador (F10)	student teacher	Persona B
/glosario	Glosario completo (F10)	student teacher	Persona B
/docente	Panel del Docente	teacher	Persona B
/docente/clase/:classCode	Detalle de clase	teacher	Persona B
*	Redirección a /mapa o /login según sesión	—	Ambos

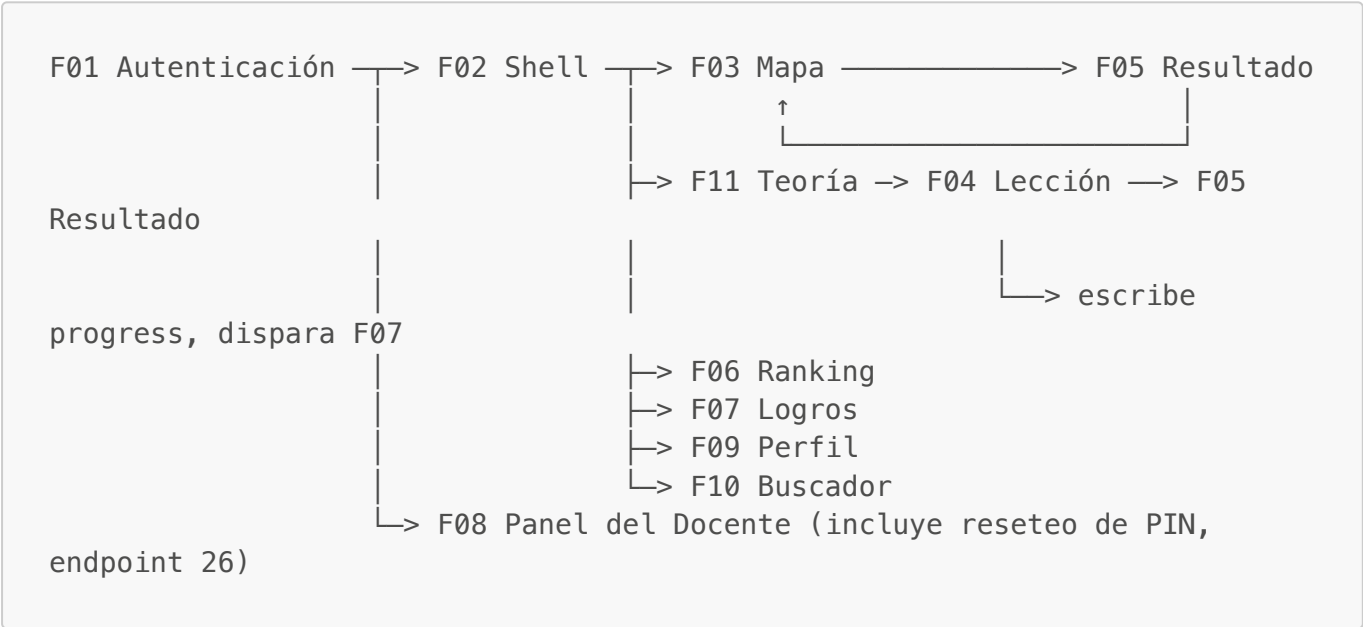
Desviación documentada. Las rutas de lección y resultado usan `:attemptId` en lugar de `:moduleId`, porque un alumno puede realizar varios intentos del mismo módulo — con `:moduleId` la URL de un resultado sería ambigua.

0.13 Mapa de features y orden de trabajo

ID	Feature	Endpoints	Colecciones	Dueño	Horas	Origen
F01	Autenticación (con PIN)	2, 3, 4, 26	users, classes	Pareja	12–16	RF1, RF2, D-05 rev.
F02	Shell de la aplicación	—	—	Pareja	6–8	Diseño p. 5, p. 13
F03	Mapa de Módulos	5, 6, 12	modules, progress	A	8–12	RF3
F04	Motor de Lección	7, 8, 9, 10	questions, moduleAttempts	B	16–22	RF4
F05	Resultado del Módulo	11	progress, userBadges	A	6–9	RF4, RF6
F06	Ranking de Clase	15	users	A	5–7	RF6

ID	Feature	Endpoints	Colecciones	Dueño	Horas	Origen
F07	Logros e Insignias	13, 14	badges, userBadges	A	4–6	RF6
F08	Panel del Docente	16–20	todas (lectura)	B	14–20	RF7
F09	Perfil de usuario	23, 24	users	A	4–6	RF9
F10	Buscador de contenido	25	glossaryTerms, modules	B	6–9	RF8
F11	Contenido teórico	27	modules	B	10–14	§4.6.3, D-16
F12	Integración Google Forms	21, 22	testResults	B	5–8	D-02 — Opcional

Núcleo obligatorio: F01 a F05, F09, F11 — comprometidas en la tesis o resueltas por decisión. F06, F07, F08 responden a RF6/RF7. F12 sigue siendo opcional (Fase 2).



Orden sugerido

Etap	Qué	Quién
1	F01 Autenticación (con PIN)	Pareja
2	F02 Shell	Pareja
3	F03 Mapa · F04 Lección, en paralelo	A · B
4	F11 Contenido teórico (redacción puede arrancar en paralelo con la etapa 3)	B
5	F05 Resultado	A
6	F09 Perfil · F10 Buscador, en paralelo	A · B
7	F07 Logros · F06 Ranking	A
8	F08 Panel del Docente (incluye endpoint 26, reseteo de PIN)	B

Etapa	Qué	Quién
9	F12 si hay tiempo	B

Única dependencia que cruza entre personas: F05 (A) lee lo que escribe F04 (B) — se resuelve con el endpoint 11, ya fijado en el contrato. A puede construir la pantalla contra un *mock* mientras B termina el motor.

0.14 Convención de commits y ramas

Elemento	Convención
Ramas	<code>feat/<feature></code> , <code>fix/<bug></code> , <code>docs/<tema></code> , <code>chore/<tarea></code>
Commits	Conventional Commits: <code>feat(modules): agregar cálculo de desbloqueo</code>
Rama protegida	<code>main</code> — sin push directo
Merge	Solo vía Pull Request con revisión aprobada del otro desarrollador
Contenido del PR	Descripción, endpoints afectados y captura de pantalla si hay cambios de UI

0.15 Protocolo de trabajo en paralelo

Situación	Procedimiento
Necesito un endpoint del otro y no está listo	Levanto un <i>mock</i> local con el shape exacto de este documento. Cuando el real esté disponible, se sustituye sin tocar el componente.
Detecto un error en el contrato	No lo corrijo unilateralmente. Aviso, acordamos, actualizamos el documento, y recién entonces implementamos.
Necesito un campo nuevo en una respuesta	Se agrega — nunca se renombra ni se elimina un campo existente. Agregar es retrocompatible; renombrar rompe la pantalla del otro.
Necesito cambiar un esquema de Mongo	Requiere acuerdo bilateral, script de migración si ya hay datos del piloto, y nota en la §6.

0.16 Checklist de arranque

Ninguno de los dos comienza su feature hasta que estos puntos estén cerrados.

#	Tarea	Responsable	Estado
1	Definir quién es Persona A y quién Persona B	Ambos	☐
2	Crear el clúster de MongoDB Atlas y compartir la <code>MONGODB_URI</code>	Uno de los dos	☐
3	Generar <code>JWT_SECRET</code> y <code>FORMS_INTEGRATION_KEY</code> , compartir por canal privado	Uno de los dos	☐

#	Tarea	Responsable	Estado
4	Implementar modelos, middlewares y utilidades compartidas (<code>models/</code> , <code>middlewares/</code> , <code>utils/</code> , <code>services/authService.js</code>)	Ambos, en pareja	☐
5	Implementar y probar los endpoints 1–4 y 26 (salud, autenticación con PIN, reseteo)	Ambos, en pareja	☐
6	Escribir y ejecutar los seeds: 6 módulos con contenido teórico , 21 preguntas, 11 insignias, 30–45 términos de glosario, 1 docente, 1 clase	Ambos	☐
7	Verificar <code>main</code> protegida y el flujo de Pull Request con revisión cruzada	Ambos	☐

A partir del punto 7, el trabajo en paralelo puede comenzar.

PARTE 1 — API por feature

Los endpoints de cada feature, agrupados en el orden de trabajo: primero autenticación y shell (compartidos), después el núcleo del alumno, después el panel del docente, después los extras. Dentro de cada feature, el shape exacto de request, response y errores es el contrato — no se improvisa.

F01 — Autenticación · Endpoints 2, 3, 4, 26

2. `POST /auth/student/login`

Endpoint unificado de registro e ingreso. Si el `studentCode` no existe en la clase, crea el alumno y **fija el PIN que envía en este primer request**; si existe, lo autentica verificando nombre y PIN.

Request

```
{
  "fullName": "Ana Rodríguez",
  "studentCode": "EC02025-01",
  "classCode": "EC02025",
  "pin": "4821"
}
```

Campo	Tipo	Requerido	Validación
<code>fullName</code>	string	Sí	3–80 caracteres. Solo letras, espacios, apóstrofes y guiones. Se normaliza a <i>Title Case</i> y se colapsan espacios múltiples.
<code>studentCode</code>	string	Sí	3–20 caracteres alfanuméricos y guiones. Se normaliza a mayúsculas.
<code>classCode</code>	string	Sí	4–12 caracteres alfanuméricos. Se normaliza a mayúsculas.

Campo	Tipo	Requerido	Validación
<code>pin</code>	string	Sí	Exactamente 4 dígitos (<code>^\d{4}\$</code>). En el primer ingreso, lo define el alumno. En los siguientes, debe coincidir con el registrado.

Respuesta **201 Created** (alumno nuevo) / **200 OK** (alumno existente)

```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "user": {
    "id": "66b1f0a2c3d4e5f6a7b8c9d0",
    "role": "student",
    "fullName": "Ana Rodríguez",
    "classCode": "EC02025",
    "totalXp": 0,
    "weeklyXp": 0,
    "currentStreak": 0,
    "isNewUser": true
  },
  "class": {
    "code": "EC02025",
    "name": "3.º Curso – Colegio X",
    "isPretestEnabled": false,
    "isPosttestEnabled": false,
    "pretestFormUrl": null,
    "posttestFormUrl": null
  }
}
```

Errores

HTTP	code	Situación
400	<code>VALIDATION_ERROR</code>	Algún campo incumple su validación, incluido un <code>pin</code> que no son 4 dígitos.
404	<code>INVALID_CLASS_CODE</code>	La clase no existe o tiene <code>isActive: false</code> .
409	<code>STUDENT_CODE_TAKEN</code>	El <code>studentCode</code> existe en la clase con un <code>fullName</code> distinto.
401	<code>INVALID_PIN</code>	El <code>studentCode</code> y el <code>fullName</code> coinciden con un alumno existente, pero el <code>pin</code> enviado no coincide con el registrado.

Regla de coincidencia. Si `studentCode` ya existe en la clase, primero se compara `fullName` normalizado — si no coincide, `STUDENT_CODE_TAKEN` (puede ser un compañero que tipeó mal el código). Si el nombre coincide, se verifica el `pin` contra `pinHash` con `bcrypt` — si no coincide, `INVALID_PIN`. Solo si ambos coinciden se emite el token.

26. PATCH /teacher/classes/:classCode/students/:studentCode/reset-pin

El docente genera un PIN nuevo para un alumno que lo olvidó. **Request sin cuerpo** — el servidor genera el PIN.

Respuesta 200 OK

```
{
  "studentCode": "EC02025-01",
  "fullName": "Ana Rodríguez",
  "newPin": "7093"
}
```

El PIN se devuelve en texto plano una sola vez, en esta respuesta. No queda almacenado en texto plano en ningún lado ni se puede volver a consultar — si el docente lo pierde antes de comunicárselo al alumno, hay que generar otro. Comunicarlo verbalmente, nunca por escrito en un canal inseguro.

Errores

HTTP	code	Situación
403	NOT_RESOURCE_OWNER	La clase pertenece a otro docente.
404	CLASS_NOT_FOUND	El <code>classCode</code> no existe.
404	USER_NOT_FOUND	El <code>studentCode</code> no existe en esa clase.

3. POST /auth/teacher/login

Request

```
{ "email": "docente@colegio.edu.py", "password": "....." }
```

Campo	Validación
email	Formato de email válido. Se normaliza a minúsculas.
password	Mínimo 8 caracteres.

Respuesta 200 OK

```
{
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": {
    "id": "66b1f0a2c3d4e5f6a7b8c9d1",
    "role": "teacher",
    "fullName": "José Souberlich",
    "email": "docente@colegio.edu.py",

```



```
    "classCode": null
  }
}
```

Errores

HTTP	code
400	VALIDATION_ERROR
401	INVALID_CREDENTIALS — mismo mensaje para email inexistente y contraseña incorrecta, para no revelar qué emails están registrados.

Las cuentas de docente se crean por script de seed, no por endpoint público. El piloto involucra un único docente.

4. GET /auth/me

Devuelve el perfil vigente del portador del token. Se usa al montar la aplicación para rehidratar el estado sin confiar en datos guardados en el navegador.

Respuesta 200 — rol student

```
{
  "id": "66b1f0a2c3d4e5f6a7b8c9d0",
  "role": "student",
  "fullName": "Ana Rodríguez",
  "classCode": "EC02025",
  "totalXp": 350,
  "weeklyXp": 150,
  "currentStreak": 3,
  "longestStreak": 5,
  "completedModules": 2,
  "totalModules": 6,
  "badgeCount": 3,
  "class": {
    "code": "EC02025",
    "name": "3.º Curso – Colegio X",
    "isPretestEnabled": true,
    "isPosttestEnabled": false,
    "pretestFormUrl": "https://forms.gle/xxxx",
    "posttestFormUrl": null
  }
}
```

Respuesta 200 — rol teacher

```
{
  "id": "66b1f0a2c3d4e5f6a7b8c9d1",
  "role": "teacher",
  "fullName": "José Souberlich",
  "email": "docente@colegio.edu.py",
  "classCode": null,
  "classCount": 1
}
```

Errores: 401 MISSING_TOKEN · 401 INVALID_TOKEN · 401 EXPIRED_TOKEN · 404 USER_NOT_FOUND

F02 — Shell de la aplicación · Sin endpoints propios

Consume **GET /auth/me** (endpoint 4, ya definido en F01) para poblar la cabecera con nombre, XP y racha.

F03 — Mapa de Módulos · Endpoints 5, 6, 12

5. **GET /modules**

Alimenta la pantalla del Mapa de Módulos por completo, en una sola petición. Incluye el estado de progreso del alumno del token — no se pasa ningún **userId**.

Respuesta 200

```
{
  "modules": [
    {
      "id": "66b2a1...",
      "order": 1,
      "slug": "ahorro",
      "title": "Ahorro",
      "description": "Fondo de emergencia, ahorro automático y metas",
      "keyConcepts": ["Fondo de emergencia", "Ahorro automático", "Metas"],
      "questionCount": 2,
      "xpReward": 100,
      "status": "completed",
      "xpEarned": 100,
      "bestCorrectAnswers": 2,
      "isPerfect": true,
      "attemptCount": 1,
      "firstCompletedAt": "2026-09-16T13:20:11.000Z",
      "activeAttemptId": null
    },
    {
      "id": "66b2a3...",
      "order": 3,
```

```

    "slug": "credito",
    "title": "Crédito y deuda",
    "description": "Tasa de interés, historial crediticio y método
avalancha",
    "keyConcepts": ["Tasa de interés", "Historial crediticio", "Método
avalancha"],
    "questionCount": 3,
    "xpReward": 150,
    "status": "available",
    "xpEarned": 0,
    "bestCorrectAnswers": 0,
    "isPerfect": false,
    "attemptCount": 0,
    "firstCompletedAt": null,
    "activeAttemptId": null
  },
  {
    "id": "66b2a4...",
    "order": 4,
    "slug": "interes-compuesto",
    "title": "Interés compuesto",
    "description": "Capitalización, tiempo y estafas de inversión",
    "keyConcepts": ["Capitalización", "Tiempo", "Estafas de inversión"],
    "questionCount": 4,
    "xpReward": 200,
    "status": "locked",
    "xpEarned": 0,
    "bestCorrectAnswers": 0,
    "isPerfect": false,
    "attemptCount": 0,
    "firstCompletedAt": null,
    "activeAttemptId": null,
    "unlockRequirement": "Completá el módulo 3"
  }
],
"summary": {
  "totalModules": 6,
  "completedModules": 2,
  "totalXp": 350,
  "weeklyXp": 150,
  "currentStreak": 3,
  "nextModuleOrder": 3
}
}

```

Contrato de campos por módulo

Campo	Tipo	Notas
status	enum	"completed" "available" "locked". Calculado por el servidor, nunca por el cliente.

Campo	Tipo	Notas
<code>activeAttemptId</code>	string null	Si no es <code>null</code> , hay un intento en curso: la UI ofrece "Continuar" en lugar de "Jugar".
<code>unlockRequirement</code>	string	Presente solo si <code>status === "locked"</code> . Texto listo para renderizar.

Regla de desbloqueo. El módulo de `order` N está `available` si y solo si el módulo de `order` N-1 tiene `status: "completed"`. El módulo de `order` 1 siempre está `available`. La progresión es estrictamente lineal.

Contrato de orden. El array viene ordenado por `order` ascendente. El frontend no reordena.

6. GET /modules/:moduleId

Detalle de un módulo, sin las preguntas. Se usa en la pantalla de confirmación previa al inicio del intento.

Respuesta 200

```
{
  "id": "66b2a3...",
  "order": 3,
  "slug": "credito",
  "title": "Crédito y deuda",
  "description": "Tasa de interés, historial crediticio y método
avalancha",
  "keyConcepts": ["Tasa de interés", "Historial crediticio", "Método
avalancha"],
  "questionCount": 3,
  "xpReward": 150,
  "xpPerCorrectAnswer": 50,
  "livesPerAttempt": 3,
  "status": "available",
  "activeAttemptId": null,
  "badge": {
    "code": "SIN_DEUDAS",
    "name": "Sin deudas",
    "emoji": "🟢",
    "isUnlocked": false
  }
}
```

Errores: 400 MALFORMED_ID · 404 MODULE_NOT_FOUND

12. GET /me/progress

Respuesta 200

```
{
  "totalXp": 500,
  "weeklyXp": 300,
  "currentStreak": 3,
  "longestStreak": 5,
  "lastActivityDate": "2026-09-20",
  "completedModules": 3,
  "totalModules": 6,
  "completionPercentage": 50,
  "badgeCount": 5,
  "totalBadges": 11,
  "totalTimeSpentSeconds": 842,
  "totalAttempts": 4,
  "perfectModules": 2
}
```

F04 — Motor de Lección · Endpoints 7, 8, 9, 10

7. POST /modules/:moduleId/attempts

Inicia un intento y devuelve la primera situación. **Request sin cuerpo.**

Respuesta 201 Created

```
{
  "attempt": {
    "id": "66c0d1...",
    "moduleId": "66b2a3...",
    "moduleTitle": "Crédito y deuda",
    "status": "in_progress",
    "livesRemaining": 3,
    "totalLives": 3,
    "currentQuestionOrder": 1,
    "totalQuestions": 3,
    "correctCount": 0,
    "startedAt": "2026-09-20T15:02:00.000Z"
  },
  "question": {
    "id": "66b3f1...",
    "order": 1,
    "statement": "Tenés una deuda en tarjeta al 28 % anual y un préstamo personal al 18 % anual. ¿Cuál conviene pagar primero?",
    "options": [
      { "key": "A", "text": "La tarjeta de crédito, porque tiene la tasa más alta" },
      { "key": "B", "text": "El préstamo personal, porque el monto es mayor" },
      { "key": "C", "text": "La deuda de cuota más baja, para sacarla rápido" }
    ]
  }
}
```

```
    ]
  }
}
```

Garantía de seguridad. El objeto `question` no contiene `correctOptionKey`, `explanation` ni `conceptTag`. Esta proyección es obligatoria y no negociable: si esos campos llegan al cliente, cualquier alumno puede leerlos desde las herramientas de desarrollo y los datos del piloto quedan invalidados.

Errores

HTTP	code	Situación
400	<code>MALFORMED_ID</code>	<code>moduleId</code> no es un <code>ObjectId</code> válido.
404	<code>MODULE_NOT_FOUND</code>	El módulo no existe o tiene <code>isPublished: false</code> .
409	<code>MODULE_LOCKED</code>	El módulo anterior no está completado.
409	<code>ATTEMPT_ALREADY_ACTIVE</code>	Ya existe un intento <code>in_progress</code> . La respuesta incluye <code>details: [{ field: "attemptId", issue: "<id>" }]</code> para que el frontend redirija a la sesión existente.

8. GET /attempts/:attemptId

Recupera el estado de un intento en curso. Permite retomar la lección tras un refresco de página o un corte de conexión — escenario probable en un aula.

Respuesta 200

```
{
  "attempt": {
    "id": "66c0d1...",
    "moduleId": "66b2a3...",
    "moduleTitle": "Crédito y deuda",
    "status": "in_progress",
    "livesRemaining": 2,
    "totalLives": 3,
    "currentQuestionOrder": 2,
    "totalQuestions": 3,
    "correctCount": 1,
    "startedAt": "2026-09-20T15:02:00.000Z"
  },
  "question": {
    "id": "66b3f2...",
    "order": 2,
    "statement": "...",
    "options": [
      { "key": "A", "text": "..." },
      { "key": "B", "text": "..." },
      { "key": "C", "text": "..." }
    ]
  }
}
```

```
    ]
  }
}
```

Si `attempt.status !== "in_progress"`, el campo `question` viene en `null` y el frontend redirige a `/resultado/:attemptId`.

Errores: 404 ATTEMPT_NOT_FOUND · 403 NOT_RESOURCE_OWNER

9. POST /attempts/:attemptId/answers

Endpoint central de la mecánica de juego. Corrige la respuesta, actualiza vidas y XP, evalúa insignias y devuelve la siguiente situación — todo en una petición.

Request

```
{
  "questionId": "66b3f1...",
  "selectedOptionKey": "A",
  "timeToAnswerSeconds": 14
}
```

Campo	Tipo	Requerido	Validación
<code>questionId</code>	string	Sí	ObjectId válido. Debe corresponder a <code>currentQuestionOrder</code> .
<code>selectedOptionKey</code>	string	Sí	"A" "B" "C".
<code>timeToAnswerSeconds</code>	number	Sí	Entero ≥ 0 . Medido por el cliente. Se acota a [0, 600] en el servidor.

Respuesta 200 — quedan situaciones por responder

```
{
  "feedback": {
    "isCorrect": true,
    "correctOptionKey": "A",
    "selectedOptionKey": "A",
    "explanation": "El método avalancha prioriza la deuda de mayor tasa. Pagar primero la tarjeta al 28 % reduce más el costo total de los intereses.",
    "xpAwarded": 50
  },
  "attempt": {
    "id": "66c0d1...",
    "status": "in_progress",
    "livesRemaining": 3,
  }
}
```

```

    "currentQuestionOrder": 2,
    "totalQuestions": 3,
    "correctCount": 1
  },
  "nextQuestion": {
    "id": "66b3f2...",
    "order": 2,
    "statement": "...",
    "options": [
      { "key": "A", "text": "..." },
      { "key": "B", "text": "..." },
      { "key": "C", "text": "..." }
    ]
  },
  "isFinished": false
}

```

Respuesta 200 — el intento finaliza con esta respuesta

```

{
  "feedback": {
    "isCorrect": true,
    "correctOptionKey": "C",
    "selectedOptionKey": "C",
    "explanation": "...",
    "xpAwarded": 50
  },
  "attempt": {
    "id": "66c0d1...",
    "status": "completed",
    "livesRemaining": 3,
    "currentQuestionOrder": 3,
    "totalQuestions": 3,
    "correctCount": 3
  },
  "nextQuestion": null,
  "isFinished": true,
  "resultUrl": "/api/v1/attempts/66c0d1.../result"
}

```

Contrato de comportamiento

Situación	<code>attempt.status</code> resultante
Respuesta correcta y quedan preguntas	<code>in_progress</code>
Respuesta incorrecta, <code>livesRemaining > 0</code>	<code>in_progress</code>
Respuesta incorrecta y <code>livesRemaining</code> llega a 0	<code>failed</code> — el intento se cierra inmediatamente
Se respondió la última pregunta con vidas restantes	<code>completed</code>

- `xpAwarded` es `50` si la respuesta es correcta **y** es la primera finalización del módulo; en cualquier otro caso es `0`.
- `correctOptionKey` y `explanation` se devuelven **siempre**, acertada o no la respuesta. Es una decisión pedagógica explícita del diseño: el refuerzo del concepto es el mecanismo por el cual la gamificación produce retención, que es la hipótesis del TCC.

Errores

HTTP	code	Situación
400	<code>VALIDATION_ERROR</code>	<code>selectedOptionKey</code> fuera de A/B/C.
403	<code>NOT_RESOURCE_OWNER</code>	El intento pertenece a otro alumno.
404	<code>ATTEMPT_NOT_FOUND</code>	—
422	<code>ATTEMPT_ALREADY_FINISHED</code>	El intento ya está cerrado.
422	<code>QUESTION_ALREADY_ANSWERED</code>	El <code>questionId</code> no coincide con <code>currentQuestionOrder</code> . Cubre el doble clic y el reenvío.

10. POST /attempts/:attemptId/abandon

El alumno cierra la lección con la X. **Request sin cuerpo.**

Respuesta 200

```
{
  "attempt": {
    "id": "66c0d1...",
    "status": "abandoned",
    "correctCount": 1,
    "totalQuestions": 3,
    "durationSeconds": 47,
    "finishedAt": "2026-09-20T15:02:47.000Z"
  },
  "message": "Tu progreso quedó guardado. Podés volver a intentarlo cuando quieras."
}
```

Un intento abandonado **no otorga XP ni insignias**, pero incrementa `progress.attemptCount` y queda registrado para el análisis de abandono del Capítulo 5.

Errores: `404 ATTEMPT_NOT_FOUND` · `403 NOT_RESOURCE_OWNER` · `422 ATTEMPT_ALREADY_FINISHED`

F05 — Resultado del Módulo · Endpoint 11

11. GET /attempts/:attemptId/result

Alimenta la pantalla de Resultado por completo. Se puede consultar cuantas veces se quiera; es idempotente y de solo lectura.

Respuesta 200

```
{
  "attempt": {
    "id": "66c0d1...",
    "status": "completed",
    "correctCount": 3,
    "totalQuestions": 3,
    "accuracyPercentage": 100,
    "isPerfect": true,
    "livesRemaining": 3,
    "durationSeconds": 172,
    "startedAt": "2026-09-20T15:02:00.000Z",
    "finishedAt": "2026-09-20T15:04:52.000Z"
  },
  "module": {
    "id": "66b2a3...",
    "order": 3,
    "title": "Crédito y deuda",
    "xpReward": 150
  },
  "rewards": {
    "xpEarned": 150,
    "isFirstCompletion": true,
    "totalXpAfter": 500,
    "badgesUnlocked": [
      { "code": "SIN_DEUDAS", "name": "Sin deudas", "emoji": "👍",
        "description": "Completá el módulo 3 (Crédito y deuda)" },
      { "code": "RESPUESTA_PERFECTA", "name": "Respuesta perfecta",
        "description": "Obtené el 100 % en cualquier módulo" }
    ]
  },
  "ranking": {
    "position": 3,
    "totalStudents": 6,
    "positionsGained": 1
  },
  "nextModule": {
    "id": "66b2a4...",
    "order": 4,
    "title": "Interés compuesto",
    "isUnlocked": true
  }
}
```

Campo	Notas
<code>accuracyPercentage</code>	Entero 0–100. Redondeo estándar.

Campo	Notas
<code>isFirstCompletion</code>	<code>false</code> en repeticiones. La UI atenúa la celebración y muestra <code>xpEarned: 0</code> .
<code>ranking.positionsGained</code>	Diferencia respecto a la posición previa al intento. Puede ser <code>0</code> o negativa.
<code>nextModule</code>	<code>null</code> si se completó el módulo 6.
<code>rewards.badgesUnlocked</code>	Array vacío si no hubo insignias nuevas. Nunca <code>null</code> .

Errores: `404 ATTEMPT_NOT_FOUND` · `403 NOT_RESOURCE_OWNER`

F06 — Ranking de Clase · Endpoint 15

15. GET /ranking

Query parameters

Parámetro	Tipo	Default	Valores	Descripción
<code>scope</code>	string	<code>weekly</code>	<code>weekly</code> <code>alltime</code>	Ordena por <code>weeklyXp</code> o por <code>totalXp</code> .
<code>limit</code>	number	<code>20</code>	1–100	Cantidad de posiciones.

Respuesta `200`

```
{
  "scope": "weekly",
  "classCode": "EC02025",
  "weekStartsAt": "2026-09-14T03:00:00.000Z",
  "weekEndsAt": "2026-09-21T02:59:59.999Z",
  "entries": [
    { "position": 1, "userId": "66b1...a1", "fullName": "Ana Rodríguez",
      "xp": 510, "completedModules": 4, "isCurrentUser": false },
    { "position": 2, "userId": "66b1...a2", "fullName": "Luis Martínez",
      "xp": 420, "completedModules": 3, "isCurrentUser": false },
    { "position": 3, "userId": "66b1...a3", "fullName": "Sofía Vera", "xp":
      350, "completedModules": 3, "isCurrentUser": false },
    { "position": 4, "userId": "66b1...a4", "fullName": "Martín Vega",
      "xp": 300, "completedModules": 3, "isCurrentUser": true }
  ],
  "currentUser": {
    "position": 4,
    "xp": 300,
    "totalStudents": 6
  }
}
```

Regla	Definición
Alcance	Solo alumnos con el mismo <code>classCode</code> que el solicitante e <code>isActive: true</code> .
Desempate	A igual XP, gana quien lo alcanzó primero (<code>updatedAt</code> ascendente). Empates estables entre peticiones.
<code>isCurrentUser</code>	El frontend usa este booleano para resaltar la fila en verde. No compara IDs por su cuenta.
<code>currentUser</code>	Siempre presente, incluso si el alumno queda fuera del <code>limit</code> . Permite mostrar "vos estás en el puesto 12".
Semana	Lunes 00:00 a domingo 23:59:59 en UTC-3 (hora de Paraguay) .

Reinicio semanal. No se usa un cron job. En cada lectura o escritura de XP, si `user.weeklyXpResetAt` es anterior al lunes de la semana en curso, `weeklyXp` se pone en 0 y se actualiza `weeklyXpResetAt`. Este enfoque perezoso es correcto, no requiere infraestructura adicional y sobrevive a los reinicios del plan gratuito de Render — donde un cron job no correría de forma confiable.

F07 — Logros e Insignias · Endpoints 13, 14

13. GET /me/badges

Respuesta 200

```
{
  "badges": [
    {
      "code": "PRIMER_AHORRO",
      "name": "Primer ahorro",
      "emoji": "🐱",
      "description": "Completá el módulo 1 (Ahorro)",
      "displayOrder": 1,
      "isUnlocked": true,
      "unlockedAt": "2026-09-16T13:20:11.000Z"
    },
    {
      "code": "VELOCISTA",
      "name": "Velocista",
      "emoji": "<",
      "description": "Completá un módulo en menos de 3 minutos",
      "displayOrder": 11,
      "isUnlocked": false,
      "unlockedAt": null
    }
  ],
  "unlockedCount": 5,
}
```

```
"totalCount": 11
}
```

El array incluye **las 11 insignias**, obtenidas y no obtenidas, ordenadas por `displayOrder`. La pantalla de Logros muestra las bloqueadas en gris con su requisito visible, según el diseño aprobado. El frontend no filtra ni reordena.

14. GET /badges

Catálogo puro, sin estado de usuario. Mismos campos que el anterior sin `isUnlocked` ni `unlockedAt`. Útil para vistas informativas y para el panel del docente.

F08 — Panel del Docente · Endpoints 16–20

16. GET /teacher/classes

```
{
  "classes": [
    {
      "code": "EC02025",
      "name": "3.º Curso – Colegio X",
      "studentCount": 6,
      "isActive": true,
      "isPretestEnabled": true,
      "isPosttestEnabled": false,
      "createdAt": "2026-09-01T12:00:00.000Z"
    }
  ],
  "total": 1
}
```

17. GET /teacher/classes/:classCode/students

Query parameters

Parámetro	Default	Valores
<code>sortBy</code>	<code>totalXp</code>	<code>totalXp</code> <code>fullName</code> <code>completedModules</code> <code>lastActivityDate</code>
<code>order</code>	<code>desc</code>	<code>asc</code> <code>desc</code>

Respuesta 200

```
{
  "classCode": "EC02025",
  "students": [
    {
```

```

    "id": "66b1...a1",
    "fullName": "Ana Rodríguez",
    "studentCode": "EC02025-01",
    "totalXp": 510,
    "weeklyXp": 350,
    "completedModules": 4,
    "totalModules": 6,
    "completionPercentage": 67,
    "currentStreak": 5,
    "badgeCount": 6,
    "totalTimeSpentSeconds": 1240,
    "totalAttempts": 6,
    "lastActivityDate": "2026-09-20",
    "isActive": true
  }
],
"total": 6
}

```

`studentCode` se expone **únicamente** en endpoints de rol docente. Nunca aparece en el ranking ni en ninguna respuesta consumida por un alumno.

18. GET /teacher/classes/:classCode/stats

Alimenta las tarjetas de estadísticas del panel.

```

{
  "classCode": "EC02025",
  "studentCount": 6,
  "activeToday": 4,
  "activeThisWeek": 6,
  "averageXp": 480,
  "averageCompletedModules": 2.3,
  "averageTimePerSessionSeconds": 208,
  "totalAttempts": 34,
  "completionRate": 82.4,
  "moduleBreakdown": [
    { "order": 1, "title": "Ahorro", "completedBy": 6, "averageAccuracy": 91.7, "averageDurationSeconds": 118 },
    { "order": 2, "title": "Presupuesto", "completedBy": 5, "averageAccuracy": 80.0, "averageDurationSeconds": 165 },
    { "order": 3, "title": "Crédito y deuda", "completedBy": 3, "averageAccuracy": 66.7, "averageDurationSeconds": 194 }
  ]
}

```

Campo	Definición precisa
<code>activeToday</code>	Alumnos con <code>lastActivityDate</code> igual a hoy (UTC-3).

Campo	Definición precisa
completionRate	$\text{intentos completed} / (\text{completed} + \text{failed} + \text{abandoned}) \times 100$.
averageAccuracy	Media de accuracyPercentage de los intentos finalizados del módulo.
moduleBreakdown	Insumo directo para el análisis de dificultad por concepto del Capítulo 5.

19. PATCH /teacher/classes/:classCode/settings

Actualización parcial. Todos los campos son opcionales; solo se modifican los presentes.

Request

```
{
  "isPretestEnabled": true,
  "pretestFormUrl": "https://forms.gle/abc123",
  "isPosttestEnabled": false,
  "posttestFormUrl": null,
  "isActive": true
}
```

Respuesta 200: el documento de la clase actualizado (mismo shape que un elemento de GET /teacher/classes).

Errores: 400 VALIDATION_ERROR (URL malformada) · 403 NOT_RESOURCE_OWNER (la clase es de otro docente) · 404 CLASS_NOT_FOUND

20. GET /teacher/classes/:classCode/export

Exportación para el análisis estadístico del TCC.

Query parameters

Parámetro	Default	Valores
format	csv	csv json
include	summary	summary attempts answers

Respuesta 200 — Content-Type: text/csv; charset=utf-8, Content-Disposition: attachment; filename="ecolingo_EC02025_summary_2026-10-15.csv"

Columnas de include=summary:

```
studentCode,fullName,totalXp,completedModules,completionPercentage,perfectModules,
badgeCount,totalAttempts,totalTimeSpentSeconds,currentStreak,longestStreak,
firstActivityDate,lastActivityDate,pretestScore,posttestScore,scoreDelta
```

Las tres últimas columnas se pueblan solo si la Fase 2 está implementada; en caso contrario se emiten vacías, sin romper la estructura del archivo.

Columnas de `include=answers` — el grano más fino, base del análisis por concepto:

```
studentCode,moduleOrder,moduleTitle,attemptId,attemptStatus,questionOrder,conceptTag,selectedOptionKey,correctOptionKey,isCorrect,timeToAnswerSeconds,answeredAt
```

Codificación. El CSV se emite en UTF-8 con BOM () para que Excel en Windows interprete correctamente las tildes y la letra ñ. Sin el BOM, "Rodríguez" aparece corrupto y el anexo del TCC queda comprometido.

El endpoint 26 (reseteo de PIN) se especifica en F01, porque es parte del mecanismo de autenticación — pero su punto de entrada en la interfaz vive acá, en el panel del docente (ver Parte 3 — F08).

F09 — Perfil de usuario · Endpoints 23, 24

23. `PATCH /me`

Actualización parcial del perfil propio. Disponible para ambos roles. La identidad se deriva de `req.user.id` — no se acepta ningún identificador por parámetro ni por body (D-11).

Request — todos los campos son opcionales; solo se modifican los presentes.

```
{ "fullName": "Ana María Rodríguez" }
```

Campo	Tipo	Rol	Validación
<code>fullName</code>	string	Ambos	3–80 caracteres. Solo letras, espacios, apóstrofes y guiones. Se normaliza a <i>Title Case</i> .
<code>email</code>	string	Solo docente	Formato válido. Se normaliza a minúsculas. Debe ser único.

Un alumno **no puede** modificar `studentCode` ni `classCode`. Son la clave de cruce con los resultados de Google Forms y la pertenencia a la clase: si un alumno los cambiara a mitad del piloto, sus datos pre/post dejarían de conciliarse. Todo intento de enviarlos se ignora silenciosamente.

Respuesta 200 OK — mismo shape que `GET /auth/me` para el rol correspondiente.

Errores

HTTP	code	Situación
400	VALIDATION_ERROR	Algún campo incumple su validación.
401	MISSING_TOKEN / INVALID_TOKEN / EXPIRED_TOKEN	—
403	FORBIDDEN_ROLE	Un alumno intenta modificar email .
409	EMAIL_ALREADY_REGISTERED	Ya existe otro docente con ese email.

24. PATCH /me/password

Cambio de contraseña. **Solo rol teacher** — los alumnos no tienen contraseña (D-05).

Request

```
{ "currentPassword": ".....", "newPassword": "....." }
```

Campo	Validación
currentPassword	Requerido. Se verifica con bcrypt contra passwordHash .
newPassword	Requerido. Mínimo 8 caracteres. Debe ser distinta de currentPassword .

Respuesta 204 No Content — sin cuerpo.

Errores

HTTP	code	Situación
400	VALIDATION_ERROR	La contraseña nueva no cumple la longitud mínima o es igual a la actual.
401	INCORRECT_CURRENT_PASSWORD	La contraseña actual no coincide.
403	FORBIDDEN_ROLE	El solicitante es un alumno.

El token vigente no se invalida. Con **JWT_EXPIRES_IN=7d** y un único docente en el piloto, implementar una lista de revocación agrega complejidad sin beneficio proporcional. Queda documentado como limitación conocida y es la respuesta correcta si el tribunal pregunta.

F10 — Buscador de contenido · Endpoint 25

25. GET /search

Buscador de contenido. Cubre el requisito funcional RF8 de la tesis §4.4. Disponible para ambos roles.

Query parameters

Parámetro	Tipo	Default	Validación
<code>q</code>	string	—	Requerido. Mínimo 2 caracteres, máximo 60.
<code>type</code>	string	<code>all</code>	<code>all</code> <code>module</code> <code>glossary</code>
<code>limit</code>	number	<code>20</code>	1–50

Respuesta **200**

```
{
  "query": "interés compuesto",
  "results": [
    {
      "type": "module",
      "id": "66b2a4...",
      "title": "Interés compuesto",
      "snippet": "Capitalización, tiempo y estafas de inversión",
      "moduleOrder": 4,
      "isAccessible": false,
      "route": "/mapa"
    },
    {
      "type": "glossary",
      "id": "66e1b2...",
      "title": "Capitalización",
      "snippet": "Proceso por el cual los intereses generados se suman al capital y pasan a generar nuevos intereses.",
      "moduleOrder": 4,
      "isAccessible": true,
      "route": "/glosario/capitalizacion"
    }
  ],
  "total": 2
}
```

Campo	Notas
<code>type</code>	<code>"module"</code> <code>"glossary"</code> . Permite al frontend agrupar los resultados.
<code>snippet</code>	Fragmento listo para mostrar. Máximo 160 caracteres, truncado por el servidor.
<code>isAccessible</code>	<code>false</code> si es un módulo que el alumno todavía no desbloqueó. La UI lo muestra atenuado y sin enlace. Para docentes siempre es <code>true</code> .
<code>route</code>	Ruta del frontend a la que navegar. Evita que el cliente construya URLs.

Reglas de búsqueda

Regla	Definición
Fuentes	<code>glossaryTerms</code> (término, definición, palabras clave) y <code>modules</code> (título, descripción, <code>keyConcepts</code>).
Exclusión	<code>questions</code> nunca se indexa (D-14).
Insensibilidad	La comparación ignora mayúsculas y tildes: buscar <code>interes</code> encuentra <code>interés</code> .
Orden	Coincidencias en <code>glossaryTerms</code> primero, luego <code>modules</code> . Dentro de cada grupo, por puntaje de relevancia del índice de texto.
Sin resultados	<code>200</code> con <code>results: []</code> y <code>total: 0</code> . No es un <code>404</code> .

Errores

HTTP	code	Situación
400	<code>SEARCH_QUERY_TOO_SHORT</code>	<code>q</code> tiene menos de 2 caracteres o está ausente.
400	<code>VALIDATION_ERROR</code>	<code>type</code> o <code>limit</code> fuera de rango.

27. GET /modules/:moduleId/content

Devuelve el contenido teórico del módulo — la pantalla que se muestra **antes** de las preguntas. Cubre la resolución de F11 (D-16).

Respuesta `200`

```
{
  "moduleId": "66b2a3...",
  "moduleOrder": 3,
  "title": "Crédito y deuda",
  "theoryContent": "## ¿Qué es el crédito?\n\nEl crédito es la posibilidad de acceder a dinero o bienes hoy, a cambio de devolverlos más adelante con un interés...",
  "estimatedReadingMinutes": 3
}
```

Campo	Notas
<code>theoryContent</code>	Markdown. El frontend lo renderiza con un componente de Markdown ya sanitizado — nunca <code>dangerouslySetInnerHTML</code> directo.
<code>estimatedReadingMinutes</code>	Informativo, se muestra en la pantalla ("Lectura de ~3 min").

Errores

HTTP	code	Situación
400	<code>MALFORMED_ID</code>	<code>moduleId</code> no es un ObjectId válido.
404	<code>MODULE_NOT_FOUND</code>	El módulo no existe.
409	<code>MODULE_LOCKED</code>	El módulo anterior no está completado — no se puede leer teoría de un módulo bloqueado.

F12 — Integración Google Forms · Endpoints 21, 22

Estos dos endpoints se implementan **solo si el cronograma lo permite**, después del núcleo y de las pantallas extra. El pre-test y el post-test se aplican por Google Forms desde el inicio; esta integración únicamente automatiza el ingreso de los resultados a la base y su visualización en el panel. Si no se implementa, los datos se exportan manualmente desde Google Sheets a CSV, sin ningún impacto sobre el resto del sistema.

21. `POST /integrations/forms/test-results`

Invocado por un Google Apps Script asociado a la planilla de respuestas, en el disparador `onFormSubmit`.

Autenticación: header `X-Integration-Key: <FORMS_INTEGRATION_KEY>`. No usa JWT — el llamante es una máquina, no un usuario.

Request

```
{
  "studentCode": "EC02025-01",
  "classCode": "EC02025",
  "testType": "pretest",
  "score": 6,
  "totalQuestions": 10,
  "submittedAt": "2026-09-05T14:12:00.000Z",
  "answers": [
    { "questionNumber": 1, "conceptTag": "interes-compuesto", "isCorrect": true },
    { "questionNumber": 2, "conceptTag": "ahorro", "isCorrect": false }
  ]
}
```

Respuesta `201 Created`

```
{
  "id": "66d0e1...",
  "studentCode": "EC02025-01",
  "userId": "66b1...a1",
  "matched": true,
  "testType": "pretest",
}
```

```

"score": 6,
"totalQuestions": 10,
"scorePercentage": 60
}

```

matched: false con **userId: null** indica que el **studentCode** del formulario no coincide con ningún alumno registrado. El registro **se guarda igualmente** — el dato no se pierde y la conciliación puede hacerse después. Esto es deliberado: perder una respuesta del piloto es un costo mucho mayor que tener un registro huérfano.

Errores: **400 VALIDATION_ERROR · 401 INVALID_INTEGRATION_KEY · 409** si ya existe un resultado para ese (**studentCode**, **testType**) — el endpoint es idempotente por diseño y devuelve el registro existente en lugar de duplicarlo.

Apps Script de referencia:

```

function onFormSubmit(e) {
  const values = e.namedValues;

  UrlFetchApp.fetch('https://<subdominio>.onrender.com/api/v1/integrations/forms/test-results', {
    method: 'post',
    contentType: 'application/json',
    headers: { 'X-Integration-Key':
PropertiesService.getScriptProperties().getProperty('KEY') },
    payload: JSON.stringify({
      studentCode: values['Código de alumno'][0],
      classCode:   values['Código de clase'][0],
      testType:    'pretest',
      score:       Number(values['Puntaje'][0]),
      totalQuestions: 10,
      submittedAt: new Date().toISOString(),
    }),
    muteHttpExceptions: true,
  });
}

```

22. GET /teacher/classes/:classCode/test-results

```

{
  "classCode": "EC02025",
  "results": [
    {
      "studentCode": "EC02025-01",
      "fullName": "Ana Rodríguez",
      "matched": true,
      "pretest": { "score": 6, "totalQuestions": 10, "scorePercentage":
60, "submittedAt": "2026-09-05T14:12:00.000Z" },

```

```

    "posttest": { "score": 9, "totalQuestions": 10, "scorePercentage":
90, "submittedAt": "2026-10-15T14:20:00.000Z" },
    "scoreDelta": 3,
    "improvementPercentage": 50.0
  }
],
"summary": {
  "studentsWithBothTests": 5,
  "averagePretestPercentage": 54.0,
  "averagePosttestPercentage": 81.0,
  "averageImprovementPercentage": 50.0,
  "meetsHypothesisThreshold": true
}
}

```

Campo	Fórmula
scoreDelta	posttest.score – pretest.score
improvementPercentage	(posttest.score – pretest.score) / pretest.score × 100
meetsHypothesisThreshold	averageImprovementPercentage >= 25, umbral declarado en la hipótesis del TCC

El campo `meetsHypothesisThreshold` es un indicador de apoyo para el docente durante el piloto. La verificación formal de la hipótesis se realiza en el Capítulo 5 con la prueba estadística correspondiente, no con este cálculo.

PARTE 2 — Base de datos por feature

Los modelos de Mongoose, índices y seeds que necesita cada feature. Algunos modelos son compartidos (`Class`, `User`) y se muestran una sola vez, en la feature donde se definen por primera vez — las features que los usan después los referencian sin repetir el código.

F01 — Autenticación · Colecciones `users`, `classes`

`users`

Campo	Tipo	Requerido	Índice	Descripción
<code>_id</code>	ObjectId	auto	PK	—
<code>role</code>	String enum	Sí	sí	"student" "teacher".
<code>fullName</code>	String	Sí	—	Nombre completo. Se normaliza (trim, colapso de espacios).

Campo	Tipo	Requerido	Índice	Descripción
<code>email</code>	String null	Solo docente	único disperso	Identificador de login del docente. Minúsculas. <code>null</code> en alumnos.
<code>passwordHash</code>	String null	Solo docente	—	bcrypt, 10 rondas. <code>null</code> en alumnos. Nunca se serializa.
<code>studentCode</code>	String null	Solo alumno	único compuesto	Identificador único del alumno dentro de su clase.
<code>classCode</code>	String null	Solo alumno	sí	Clase a la que pertenece. <code>null</code> en docentes.
<code>pinHash</code>	String null	Solo alumno	—	bcrypt del PIN de 4 dígitos que el alumno elige en su primer ingreso. <code>null</code> en docentes. Nunca se serializa. Ver 0.2.1.
<code>totalXp</code>	Number	Sí	sí (desc)	XP acumulado histórico. Default <code>0</code> .
<code>weeklyXp</code>	Number	Sí	sí (desc)	XP de la semana en curso. Base del ranking. Default <code>0</code> .
<code>weeklyXpResetAt</code>	Date	Sí	—	Inicio de la semana de cómputo (lunes 00:00 UTC-3).
<code>currentStreak</code>	Number	Sí	—	Días consecutivos de actividad. Default <code>0</code> .
<code>longestStreak</code>	Number	Sí	—	Racha máxima histórica. Default <code>0</code> .
<code>lastActivityDate</code>	String null	Sí	—	Fecha <code>YYYY-MM-DD</code> de la última actividad.
<code>totalTimeSpentSeconds</code>	Number	Sí	—	Tiempo acumulado en lecciones.
<code>isActive</code>	Boolean	Sí	—	Permite al docente dar de baja a un alumno sin borrar sus datos. Default <code>true</code> .
<code>createdAt / updatedAt</code>	Date	auto	—	—

Índices:

```
userSchema.index({ email: 1 }, { unique: true, sparse: true });
```

```

userSchema.index({ classCode: 1, studentCode: 1 }, { unique: true, sparse: true });
userSchema.index({ classCode: 1, weeklyXp: -1 }); // ranking semanal
userSchema.index({ classCode: 1, totalXp: -1 }); // panel del docente

```

Modelo Mongoose:

```

import mongoose from 'mongoose';
const { Schema } = mongoose;

const userSchema = new Schema({
  role: {
    type: String, required: true,
    enum: { values: ['student', 'teacher'], message: 'Rol inválido: {VALUE}' },
    index: true,
  },
  fullName: { type: String, required: true, trim: true, minlength: 3,
    maxlength: 80 },

  // --- Solo docentes ---
  email: {
    type: String, default: null, lowercase: true, trim: true,
    match: [/^[^\s@]+@[^\s@]+\.[^\s@]+$/, 'Email inválido.'],
  },
  passwordHash: { type: String, default: null, select: false },

  // --- Solo alumnos ---
  studentCode: { type: String, default: null, uppercase: true, trim: true,
    maxlength: 20 },
  classCode: { type: String, default: null, uppercase: true, trim: true,
    index: true },
  pinHash: { type: String, default: null, select: false }, // bcrypt
  del PIN de 4 dígitos – ver 0.2.1

  // --- Gamificación ---
  totalXp: { type: Number, default: 0, min: 0 },
  weeklyXp: { type: Number, default: 0, min: 0 },
  weeklyXpResetAt: { type: Date, default: Date.now },
  currentStreak: { type: Number, default: 0, min: 0 },
  longestStreak: { type: Number, default: 0, min: 0 },
  lastActivityDate: { type: String, default: null }, // 'YYYY-MM-DD' en
  UTC-3

  // --- Métricas del piloto ---
  totalTimeSpentSeconds: { type: Number, default: 0, min: 0 },

  isActive: { type: Boolean, default: true },
}, { timestamps: true });

// Validación condicional por rol: reemplaza el CHECK de PostgreSQL.
userSchema.pre('validate', function (next) {

```



```

    if (this.role === 'teacher') {
      if (!this.email) return next(new Error('Un docente requiere email.'));
      if (!this.passwordHash) return next(new Error('Un docente requiere contraseña.'));
    }
    if (this.role === 'student') {
      if (!this.studentCode) return next(new Error('Un alumno requiere studentCode.'));
      if (!this.classCode) return next(new Error('Un alumno requiere classCode.'));
      if (!this.pinHash) return next(new Error('Un alumno requiere PIN.'));
    }
    next();
  });

// Índices – Parte 2, §2 de esta guía
userSchema.index({ email: 1 }, { unique: true, sparse: true });
userSchema.index({ classCode: 1, studentCode: 1 }, { unique: true, sparse: true });
userSchema.index({ classCode: 1, weeklyXp: -1 });
userSchema.index({ classCode: 1, totalXp: -1 });

export default mongoose.model('User', userSchema);

```

Cuatro detalles que importan:

select: false en **passwordHash** y en **pinHash** hace que ninguno de los dos se traiga en consultas normales. Para verificarlos hay que pedirlos explícitamente: `User.findOne({ ... }).select('+pinHash')`. Es una segunda capa de defensa, sobre el **delete** del transform global.

sparse: true en los índices únicos es indispensable. Sin él, el índice único sobre **email** trataría a todos los alumnos —que tienen **email: null**— como duplicados entre sí.

pre('validate') es donde vive la validación condicional por rol, incluyendo que todo alumno tenga **pinHash** antes de guardarse.

Generar y verificar el PIN, en el servicio de autenticación:

```

// backend/src/services/authService.js
import bcrypt from 'bcrypt';

const rounds = Number(process.env.BCRYPT_SALT_ROUNDS ?? 10);

export const hashPin = (pin) => bcrypt.hash(pin, rounds);
export const verifyPin = (pin, pinHash) => bcrypt.compare(pin, pinHash);

// Usado por el endpoint 26 – genera un PIN aleatorio de 4 dígitos

```

```
export const generateRandomPin = () =>
  String(Math.floor(1000 + Math.random() * 9000));
```

classes

src/models/Class.js

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

const classSchema = new Schema({
  code: {
    type: String, required: true, unique: true,
    uppercase: true, trim: true, minlength: 4, maxlength: 12,
    match: [/^[A-Z0-9]+$/, 'El código solo admite letras y números.'],
  },
  name: { type: String, required: true, trim: true, maxlength:
120 },
  teacherId: { type: Schema.Types.ObjectId, ref: 'User', required:
true, index: true },
  isActive: { type: Boolean, default: true },
  isPretestEnabled: { type: Boolean, default: false },
  isPosttestEnabled: { type: Boolean, default: false },
  pretestFormUrl: { type: String, default: null },
  posttestFormUrl: { type: String, default: null },
}, { timestamps: true });

export default mongoose.model('Class', classSchema);
```

Nomenclatura. `mongoose.model('Class', ...)` crea la colección `classes` — Mongoose pluraliza y pasa a minúsculas automáticamente. El nombre en singular y capitalizado es el del modelo; el plural en minúsculas es el de la colección.

Seed — docente y clase

src/seeds/classroom.js

```
import bcrypt from 'bcrypt';
import { User, Class } from '../models/index.js';

export const seedClassroom = async () => {
  const email = process.env.SEED_TEACHER_EMAIL ??
'docente@ecolingo.py';
  const password = process.env.SEED_TEACHER_PASSWORD ??
'CambiarEstaClave2026';
  const rounds = Number(process.env.BCRYPT_SALT_ROUNDS ?? 10);

  let teacher = await User.findOne({ email });
  if (!teacher) {
```

```

teacher = await User.create({
  role: 'teacher',
  fullName: 'Docente Ecolingo',
  email,
  passwordHash: await bcrypt.hash(password, rounds),
});
console.log(`✅ Docente creado: ${email}`);
} else {
  console.log(`❌ Docente ya existente: ${email}`);
}

await Class.updateOne(
  { code: 'EC02025' },
  {
    $set: { name: 'Grupo piloto – 3.º Curso', teacherId: teacher._id,
isActive: true },
    $setOnInsert: { isPretestEnabled: false, isPosttestEnabled: false },
  },
  { upsert: true }
);
console.log('✅ Clase EC02025 cargada');
};

```

\$setOnInsert vs \$set. `$set` se aplica siempre; `$setOnInsert` solo cuando el documento se crea. Los flags de pre/post-test van en `$setOnInsert` para que volver a correr el seed en octubre no desactive un test que el docente ya había habilitado. Es un detalle chico con consecuencias reales durante el piloto.

Nunca dejes la contraseña por defecto en producción. Agregá a `.env`:

```

SEED_TEACHER_EMAIL=souberlich@ejemplo.edu.py
SEED_TEACHER_PASSWORD=una_clave_fuerte_y_distinta

```

F02 — Shell de la aplicación · Sin colecciones propias

F03 — Mapa de Módulos · Colecciones `modules`, `progress`

`modules`

Campo	Tipo	Requerido	Índice	Descripción
<code>_id</code>	ObjectId	auto	PK	—
<code>order</code>	Number	Sí	único	Posición en el camino de aprendizaje (1–6). Define el desbloqueo.
<code>slug</code>	String	Sí	único	Identificador legible. Ej: <code>ahorro</code> .

Campo	Tipo	Requerido	Índice	Descripción
<code>title</code>	String	Sí	—	Ej: Ahorro .
<code>description</code>	String	Sí	—	Subtítulo mostrado en el mapa.
<code>keyConcepts</code>	[String]	Sí	—	Conceptos clave.
<code>questionCount</code>	Number	Sí	—	Cantidad de situaciones del módulo.
<code>xpReward</code>	Number	Sí	—	XP total de un módulo perfecto.
<code>xpPerCorrectAnswer</code>	Number	Sí	—	XP por respuesta correcta. Valor fijo 50 .
<code>badgeCode</code>	String	Sí	—	Insignia que se desbloquea al completarlo.
<code>theoryContent</code>	String	Sí	—	Contenido teórico en Markdown, mostrado antes de las preguntas (F11 / D-16). Máximo 4000 caracteres.
<code>estimatedReadingMinutes</code>	Number	Sí	—	Minutos estimados de lectura del bloque teórico. Default 3 .
<code>isPublished</code>	Boolean	Sí	—	Permite al docente ocultar un módulo. Default true .

Modelo Mongoose (versión final, con los campos de F11 ya incorporados):

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

const moduleSchema = new Schema({
  order: { type: Number, required: true, unique: true, min: 1, max: 6 },
  slug: { type: String, required: true, unique: true, lowercase: true, trim: true },
  title: { type: String, required: true, trim: true, maxlength: 80 },
  description: { type: String, required: true, trim: true, maxlength: 200 },
  keyConcepts: { type: [String], required: true, default: [] },
  questionCount: { type: Number, required: true, min: 1 },
  xpPerCorrectAnswer: { type: Number, required: true, default: 50, min: 1 },
  xpReward: { type: Number, required: true, min: 1 },
  badgeCode: { type: String, required: true, uppercase: true },
  // --- F11 / D-16 - contenido teórico, previo a las preguntas ---
  theoryContent: { type: String, required: true, trim: true, maxlength: 4000 },
});
```

```

    estimatedReadingMinutes: { type: Number, required: true, min: 1, default:
3 },

    isPublished: { type: Boolean, default: true },
  }, { timestamps: true });

// Invariante D-06: xpReward = questionCount × xpPerCorrectAnswer
moduleSchema.pre('validate', function (next) {
  const esperado = this.questionCount * this.xpPerCorrectAnswer;
  if (this.xpReward !== esperado) {
    return next(new Error(
      `Inconsistencia de XP en el módulo ${this.order}: ` +
      `xpReward=${this.xpReward} pero
questionCount×xpPerCorrectAnswer=${esperado}.`
    ));
  }
  next();
});

export default mongoose.model('Module', moduleSchema);

```

El campo `theoryContent` es `required: true` porque F11 ya está resuelta como "se implementa" (D-16): todo módulo debe tener su bloque teórico antes de publicarse. El seed de módulos (Parte 2 — F11) tiene que traer este campo completo para los seis módulos, o el seed falla — igual que ya pasa con `xpReward`.

progress

src/models/Progress.js

```

import mongoose from 'mongoose';
const { Schema } = mongoose;

const progressSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  moduleId: { type: Schema.Types.ObjectId, ref: 'Module', required: true },
  status: {
    type: String, required: true,
    enum: ['locked', 'available', 'completed'],
    default: 'locked',
  },
  bestCorrectAnswers: { type: Number, default: 0, min: 0 },
  xpEarned: { type: Number, default: 0, min: 0 },
  attemptCount: { type: Number, default: 0, min: 0 },
  isPerfect: { type: Boolean, default: false },
  bestDurationSeconds: { type: Number, default: null, min: 0 },
  firstCompletedAt: { type: Date, default: null },
  lastAttemptAt: { type: Date, default: null },
}, { timestamps: true });

progressSchema.index({ userId: 1, moduleId: 1 }, { unique: true });

```

```
progressSchema.index({ userId: 1, status: 1 });

export default mongoose.model('Progress', progressSchema);
```

Sobre `xpEarned`. Este campo es el que implementa la regla anti-farming (D-08). El servicio de XP verifica `progress.xpEarned === 0` antes de acreditar. Si ya es mayor a cero, el intento actualiza estadísticas pero no suma puntos. Sin esta regla, un alumno repite el módulo 1 quince veces y el ranking deja de medir aprendizaje — y con él, los datos del Capítulo 5.

Sin seed. Los documentos de `progress` se crean de forma perezosa en el primer acceso al mapa — ver F03 en la Parte 3 y el riesgo correspondiente.

Seed — módulos

`src/seeds/modules.js`

```
import { Module } from '../models/index.js';

export const MODULES = [
  { order: 1, slug: 'ahorro', title: 'Ahorro',
    description: 'Fondo de emergencia, ahorro automático y metas',
    keyConcepts: ['Fondo de emergencia', 'Ahorro automático', 'Metas'],
    questionCount: 2, xpPerCorrectAnswer: 50, xpReward: 100, badgeCode:
    'PRIMER_AHORRO' },
  { order: 2, slug: 'presupuesto', title: 'Presupuesto',
    description: 'Regla 50/30/20, gastos fijos y variables',
    keyConcepts: ['Regla 50/30/20', 'Gastos fijos', 'Gastos variables'],
    questionCount: 3, xpPerCorrectAnswer: 50, xpReward: 150, badgeCode:
    'PRESUPUESTADOR' },
  { order: 3, slug: 'credito', title: 'Crédito y deuda',
    description: 'Tasa de interés, historial crediticio y método avalancha',
    keyConcepts: ['Tasa de interés', 'Historial crediticio', 'Método
    avalancha'], questionCount: 3, xpPerCorrectAnswer: 50, xpReward: 150,
    badgeCode: 'SIN_DEUDAS' },
  { order: 4, slug: 'interes-compuesto', title: 'Interés compuesto',
    description: 'Capitalización, tiempo y estafas de inversión',
    keyConcepts: ['Capitalización', 'Tiempo', 'Estafas de inversión'],
    questionCount: 4, xpPerCorrectAnswer: 50, xpReward: 200, badgeCode:
    'INVERSOR_NOVATO' },
  { order: 5, slug: 'inversion', title: 'Inversión básica',
    description: 'Riesgo, diversificación, plazo fijo y acciones',
    keyConcepts: ['Riesgo', 'Diversificación', 'Plazo fijo vs acciones'],
    questionCount: 4, xpPerCorrectAnswer: 50, xpReward: 200, badgeCode:
    'INVERSIONES' },
  { order: 6, slug: 'planificacion', title: 'Planificación financiera',
    description: 'Metas de corto, mediano y largo plazo',
    keyConcepts: ['Corto plazo', 'Mediano plazo', 'Largo plazo'],
    questionCount: 5, xpPerCorrectAnswer: 50, xpReward: 250, badgeCode:
    'PLANIFICADOR' },
];
```

```
export const seedModules = async () => {
  for (const mod of MODULES) {
    await Module.updateOne({ order: mod.order }, { $set: mod }, { upsert:
true, runValidators: true });
  }
  console.log(`✅ ${MODULES.length} módulos cargados`);
};
```

Actualizar para incluir `theoryContent` y `estimatedReadingMinutes` en cada uno de los 6 módulos — campos requeridos desde que F11 se resolvió como implementada (D-16).

F04 — Motor de Lección · Colecciones `questions`, `moduleAttempts`

`questions`

`src/models/Question.js`

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

const optionSchema = new Schema({
  key: { type: String, required: true, enum: ['A', 'B', 'C'] },
  text: { type: String, required: true, trim: true, maxLength: 200 },
}, { _id: false });

const questionSchema = new Schema({
  moduleId: { type: Schema.Types.ObjectId, ref: 'Module', required: true },
  order: { type: Number, required: true, min: 1 },
  statement: { type: String, required: true, trim: true, maxLength: 500 },
  options: {
    type: [optionSchema], required: true,
    validate: {
      validator: (v) =>
        v.length === 3 &&
        ['A', 'B', 'C'].every((k) => v.some((o) => o.key === k)),
      message: 'Cada pregunta requiere exactamente 3 opciones con claves A,
B y C.',
    },
  },
  correctOptionKey: { type: String, required: true, enum: ['A', 'B', 'C'] },
  explanation: { type: String, required: true, trim: true, maxLength:
400 },
  conceptTag: { type: String, required: true, lowercase: true, trim:
true },
}, { timestamps: true });

questionSchema.index({ moduleId: 1, order: 1 }, { unique: true });
```

```
// Proyección obligatoria para servir una pregunta al cliente.
// D-07 del Contrato Técnico: correctOptionKey y explanation NUNCA llegan
// al navegador antes de que el alumno responda.
questionSchema.statics.publicProjection = function () {
  return '-correctOptionKey -explanation -conceptTag -createdAt -
updatedAt';
};

export default mongoose.model('Question', questionSchema);
```

Uso obligatorio en el controlador:

```
const question = await Question
  .findOne({ moduleId, order })
  .select(Question.publicProjection());
```

`_id: false` en `optionSchema`. Por defecto, Mongoose asigna un `_id` a cada subdocumento embebido. Para las opciones de respuesta no aporta nada y solo agrega ruido a la respuesta JSON, así que se desactiva.

moduleAttempts

El modelo más complejo. Es la sesión de juego y, a la vez, el registro más rico para el análisis del Capítulo 5.

src/models/ModuleAttempt.js

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

const answerSchema = new Schema({
  questionId: { type: Schema.Types.ObjectId, ref: 'Question',
required: true },
  questionOrder: { type: Number, required: true, min: 1 },
  selectedOptionKey: { type: String, required: true, enum: ['A', 'B',
'C'] },
  isCorrect: { type: Boolean, required: true },
  answeredAt: { type: Date, default: Date.now },
  timeToAnswerSeconds: { type: Number, required: true, min: 0, max: 600 },
}, { _id: false });

const moduleAttemptSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  moduleId: { type: Schema.Types.ObjectId, ref: 'Module', required: true },
  status: {
    type: String, required: true,
    enum: ['in_progress', 'completed', 'failed', 'abandoned'],
    default: 'in_progress',
  },
},
```



```

    livesRemaining:      { type: Number, default: 3, min: 0, max: 3 },
    currentQuestionOrder: { type: Number, default: 1, min: 1 },
    answers:             { type: [answerSchema], default: [] },
    correctCount:        { type: Number, default: 0, min: 0 },
    xpEarned:            { type: Number, default: 0, min: 0 },
    startedAt:           { type: Date, default: Date.now },
    finishedAt:          { type: Date, default: null },
    durationSeconds:     { type: Number, default: null, min: 0 },
    badgesUnlocked:      { type: [String], default: [] },
  }, { timestamps: true });

moduleAttemptSchema.index({ userId: 1, moduleId: 1, status: 1 });
moduleAttemptSchema.index({ userId: 1, status: 1 });

// Índice único parcial: como máximo un intento activo por (alumno,
// módulo).
// Es el equivalente al CREATE UNIQUE INDEX ... WHERE de PostgreSQL.
moduleAttemptSchema.index(
  { userId: 1, moduleId: 1 },
  { unique: true, partialFilterExpression: { status: 'in_progress' } }
);

// Calcula la duración automáticamente al cerrar el intento.
moduleAttemptSchema.pre('save', function (next) {
  if (this.finishedAt && this.durationSeconds === null) {
    this.durationSeconds = Math.round((this.finishedAt - this.startedAt) /
1000);
  }
  next();
});

export default mongoose.model('ModuleAttempt', moduleAttemptSchema);

```

El índice parcial es la pieza clave. El Contrato Técnico establece que un alumno no puede tener dos intentos `in_progress` del mismo módulo. Podrías verificarlo con un `findOne` antes de crear, pero eso tiene una condición de carrera: entre la lectura y la escritura, dos peticiones simultáneas —dos pestañas abiertas, un doble clic— pueden pasar ambas la verificación.

`partialFilterExpression` hace que la **base** rechace el segundo insert. El controlador captura el error 11000 y devuelve `ATTEMPT_ALREADY_ACTIVE`. La invariante queda garantizada por el motor, no por la lógica. Esto también es material de defensa: demuestra comprensión de concurrencia, no solo de CRUD.

Seed — preguntas

`src/seeds/questions.js`

Estructura del archivo y ejemplos completos. **Persona A** escribe los módulos 1 a 4 (12 preguntas); **Persona B**, los módulos 5 y 6 (9 preguntas).

```
import { Module, Question } from '../models/index.js';

// Agrupadas por slug de módulo. Cada array debe tener exactamente
// `questionCount` elementos, en el orden en que se presentan.
export const QUESTIONS_BY_MODULE = {

  // — Módulo 1 · Ahorro – 2 preguntas · Persona A —————
  'ahorro': [
    {
      statement: 'Ganás Gs. 1.500.000 por mes. ¿Cuánto deberías tener idealmente en tu fondo de emergencia?',
      options: [
        { key: 'A', text: 'Gs. 500.000, con eso alcanza para un imprevisto' },
        { key: 'B', text: 'Entre Gs. 4.500.000 y Gs. 9.000.000, de 3 a 6 meses de ingresos' },
        { key: 'C', text: 'No hace falta si tenés tarjeta de crédito' },
      ],
      correctOptionKey: 'B',
      explanation: 'Un fondo de emergencia debe cubrir entre 3 y 6 meses de tus ingresos. Con Gs. 1.500.000 mensuales, el rango va de Gs. 4.500.000 a Gs. 9.000.000. La tarjeta no es un fondo de emergencia: es deuda que genera intereses.',
      conceptTag: 'fondo-de-emergencia',
    },
    {
      statement: 'Cobrás tu sueldo el día 30. ¿Cuál es la mejor estrategia para ahorrar?',
      options: [
        { key: 'A', text: 'Apartar el monto de ahorro apenas cobrás, antes de gastar' },
        { key: 'B', text: 'Gastar normalmente y ahorrar lo que sobre a fin de mes' },
        { key: 'C', text: 'Ahorrar solo los meses en que no tenés gastos grandes' },
      ],
      correctOptionKey: 'A',
      explanation: 'Se llama "pagarse a uno mismo primero". Si esperás a que sobre, casi nunca sobra. Apartar el ahorro apenas cobrás lo convierte en un gasto fijo más, y el resto del mes te acomodás con lo que queda.',
      conceptTag: 'ahorro-automatico',
    },
  ],

  // — Módulo 3 · Crédito y deuda – 3 preguntas · Persona A —————
  'credito': [
    {
      statement: 'Tenés una deuda en tarjeta al 28 % anual y un préstamo personal al 18 % anual. ¿Cuál conviene pagar primero?',
      options: [
        { key: 'A', text: 'La tarjeta de crédito, porque tiene la tasa más alta' },
      ],
    },
  ],
}
```

```

    { key: 'B', text: 'El préstamo personal, porque el monto es mayor'
  },
    { key: 'C', text: 'La deuda de cuota más baja, para sacarla rápido'
  },
  ],
  correctOptionKey: 'A',
  explanation: 'El método avalancha prioriza la deuda de mayor tasa.
  Pagar primero la tarjeta al 28 % reduce más el costo total en intereses que
  cancelar el préstamo al 18 %, aunque este último tenga un saldo mayor.',
  conceptTag: 'metodo-avalancha',
},
// ... 2 preguntas más
],

'presupuesto':      [ /* 3 preguntas – Persona A */ ],
'interes-compuesto': [ /* 4 preguntas – Persona A */ ],
'inversion':         [ /* 4 preguntas – Persona B */ ],
'planificacion':     [ /* 5 preguntas – Persona B */ ],
};

export const seedQuestions = async () => {
  let total = 0;

  for (const [slug, questions] of Object.entries(QUESTIONS_BY_MODULE)) {
    const mod = await Module.findOne({ slug });
    if (!mod) throw new Error(`Módulo no encontrado: ${slug}. ¿Corriste
    seedModules primero?`);

    if (questions.length !== mod.questionCount) {
      throw new Error(
        `El módulo "${slug}" declara questionCount=${mod.questionCount} ` +
        `pero el seed trae ${questions.length} preguntas.`
      );
    }

    for (const [i, q] of questions.entries()) {
      await Question.updateOne(
        { moduleId: mod._id, order: i + 1 },
        { $set: { ...q, moduleId: mod._id, order: i + 1 } },
        { upsert: true, runValidators: true }
      );
      total++;
    }
  }

  console.log(`✅ ${total} preguntas cargadas`);
};

```

Esa verificación de `questionCount` es importante. Si Persona B escribe 5 preguntas para el módulo `inversion` pero el módulo declara 4, el seed falla con un mensaje explícito. Sin la verificación, el módulo tendría una pregunta que nunca se muestra y el XP no cerraría — un bug muy difícil de rastrear después.

Criterios para escribir las preguntas

Ambos escriben contenido pedagógico, así que conviene un estándar común:

Criterio	Detalle
Contexto local	Montos en guaraníes, situaciones reconocibles para un adolescente de Asunción.
Situación, no definición	❌ "¿Qué es el interés compuesto?" → ✅ "Si depositás Gs. 1.000.000 al 10 % anual capitalizable, ¿cuánto tenés a los 3 años?"
Tres opciones plausibles	La incorrecta debe reflejar un error de razonamiento real, no ser un absurdo.
Explicación que enseña	No repitas la respuesta: explicá por qué . Es el mecanismo central de la hipótesis del TCC.
Longitud	Enunciado ≤ 500 caracteres, opciones ≤ 200, explicación ≤ 400.
conceptTag	En minúsculas y con guiones. Debe coincidir con alguno de los keyConcepts del módulo. Es lo que permite el análisis por concepto del Capítulo 5.

F05 — Resultado del Módulo · Sin colecciones nuevas

Lee **moduleAttempts** (F04), **progress** (F03), **userBadges** (F07) y **users** (F01), todos ya definidos.

F06 — Ranking de Clase · Sin colecciones nuevas

Lee **users** (F01), usando los índices { **classCode**, **weeklyXp** } y { **classCode**, **totalXp** } ya declarados en ese modelo.

F07 — Logros e Insignias · Colecciones **badges**, **userBadges**

src/models/Badge.js

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

export const CRITERIA_TYPES = [
  'MODULE_COMPLETED', 'ALL_MODULES_COMPLETED', 'PERFECT_MODULE',
  'STREAK_DAYS', 'TOP_THREE_WEEKLY', 'SPEED_RUN',
];

const badgeSchema = new Schema({
  code: { type: String, required: true, unique: true, uppercase: true, trim: true },
  name: { type: String, required: true, trim: true, maxlength: 60 },
  description: { type: String, required: true, trim: true, maxlength: 160 }
```

```
,
  emoji: { type: String, required: true },
  criteriaType: { type: String, required: true, enum: CRITERIA_TYPES },
  criteriaValue: { type: Schema.Types.Mixed, default: null },
  displayOrder: { type: Number, required: true, min: 1 },
}, { timestamps: true });

badgeSchema.index({ displayOrder: 1 });

export default mongoose.model('Badge', badgeSchema);
```

src/models/UserBadge.js

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

const userBadgeSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required:
true },
  badgeCode: { type: String, required: true, uppercase: true },
  unlockedAt: { type: Date, default: Date.now },
  sourceAttemptId: { type: Schema.Types.ObjectId, ref: 'ModuleAttempt',
default: null },
}, { timestamps: true });

// Este índice único es lo que hace idempotente a evaluateBadges().
userBadgeSchema.index({ userId: 1, badgeCode: 1 }, { unique: true });

export default mongoose.model('UserBadge', userBadgeSchema);
```

Idempotencia por índice. El servicio de insignias se ejecuta al final de cada intento y reevalúa todos los criterios. Sin este índice, un alumno que completa el módulo 3 por segunda vez recibiría la insignia **SIN_DEUDAS** dos veces. Con el índice, el segundo `insertOne` falla con el código de error 11000 (duplicado), que el servicio captura e ignora. La regla de negocio la garantiza la base, no la lógica — que es siempre más robusto.

Seed — insignias

src/seeds/badges.js

```
import { Badge } from '../models/index.js';

export const BADGES = [
  { code: 'PRIMER_AHORRO', name: 'Primer ahorro', emoji: '🐷',
description: 'Completa el módulo 1 (Ahorro)', criteriaType:
'MODULE_COMPLETED', criteriaValue: 1, displayOrder: 1 },
  { code: 'PRESUPUESTADOR', name: 'Presupuestador', emoji: '🇩🇪',
description: 'Completa el módulo 2 (Presupuesto)', criteriaType:
'MODULE_COMPLETED', criteriaValue: 2, displayOrder: 2 },
```

```

    { code: 'SIN_DEUDAS',      name: 'Sin deudas',      emoji: '🏠',
      description: 'Completá el módulo 3 (Crédito y deuda)',      criteriaType:
      'MODULE_COMPLETED',      criteriaValue: 3,      displayOrder: 3 },
    { code: 'INVERSOR_NOVATO',  name: 'Inversor novato',  emoji: '📈',
      description: 'Completá el módulo 4 (Interés compuesto)',      criteriaType:
      'MODULE_COMPLETED',      criteriaValue: 4,      displayOrder: 4 },
    { code: 'INVERSIONES',      name: 'Inversiones',      emoji: '💰',
      description: 'Completá el módulo 5 (Inversión básica)',      criteriaType:
      'MODULE_COMPLETED',      criteriaValue: 5,      displayOrder: 5 },
    { code: 'PLANIFICADOR',      name: 'Planificador',      emoji: '🎯',
      description: 'Completá el módulo 6 (Planificación)',      criteriaType:
      'MODULE_COMPLETED',      criteriaValue: 6,      displayOrder: 6 },
    { code: 'MODULOS_COMPLETOS', name: 'Módulos completos', emoji: '🏆',
      description: 'Completá los 6 módulos',      criteriaType:
      'ALL_MODULES_COMPLETED', criteriaValue: null, displayOrder: 7 },
    { code: 'RACHA_7_DIAS',      name: 'Racha de 7 días',      emoji: '🔥',
      description: 'Usá la plataforma 7 días seguidos',      criteriaType:
      'STREAK_DAYS',      criteriaValue: 7,      displayOrder: 8 },
    { code: 'RESPUESTA_PERFECTA', name: 'Respuesta perfecta', emoji: '⭐',
      description: 'Obtené el 100 % en cualquier módulo',      criteriaType:
      'PERFECT_MODULE',      criteriaValue: null, displayOrder: 9 },
    { code: 'TOP_3_PODIO',      name: 'Top 3 del podio',      emoji: '🏅',
      description: 'Quedá entre los 3 primeros del ranking semanal', criteriaType:
      'TOP_THREE_WEEKLY',      criteriaValue: null, displayOrder: 10 },
    { code: 'VELOCISTA',      name: 'Velocista',      emoji: '⚡',
      description: 'Completá un módulo en menos de 3 minutos',      criteriaType:
      'SPEED_RUN',      criteriaValue: 180,      displayOrder: 11 },
  ];

export const seedBadges = async () => {
  for (const badge of BADGES) {
    await Badge.updateOne({ code: badge.code }, { $set: badge }, { upsert:
true });
  }
  console.log(`✅ ${BADGES.length} insignias cargadas`);
};

```

Por qué `upsert` y no `insertMany`. El seed tiene que poder ejecutarse varias veces sin romperse ni duplicar. `updateOne` con `upsert: true` inserta si no existe y actualiza si existe. Es la operación idempotente equivalente a un `INSERT ... ON CONFLICT DO UPDATE` de PostgreSQL.

F08 — Panel del Docente · Sin colecciones nuevas

Lee todas las colecciones existentes. Escribe sobre `classes` (F01, endpoint 19) y sobre `users.pinHash` (F01, endpoint 26).

F09 — Perfil de usuario · Sin colecciones nuevas

Lee y escribe sobre `users`, ya definido en F01.

F10 — Buscador de contenido · Colección **glossaryTerms**

src/models/GlossaryTerm.js

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

const glossaryTermSchema = new Schema({
  term: { type: String, required: true, trim: true, maxlength: 80 },
  slug: { type: String, required: true, unique: true, lowercase: true, trim: true },
  definition: { type: String, required: true, trim: true, maxlength: 400 },
  keywords: { type: [String], default: [] },
  moduleOrder: { type: Number, default: null, min: 1, max: 6 },
  displayOrder: { type: Number, required: true, min: 1 },
}, { timestamps: true });

glossaryTermSchema.index({ slug: 1 }, { unique: true });

// Un solo índice de texto por colección. Stemming en español y pesos por campo.
glossaryTermSchema.index(
  { term: 'text', definition: 'text', keywords: 'text' },
  {
    default_language: 'spanish',
    weights: { term: 10, keywords: 5, definition: 1 },
    name: 'glossary_text_search',
  }
);

export default mongoose.model('GlossaryTerm', glossaryTermSchema);
```

Cómo se consulta (referencia para la feature F10):

```
const results = await GlossaryTerm
  .find({ $text: { $search: q } }, { score: { $meta: 'textScore' } })
  .sort({ score: { $meta: 'textScore' } })
  .limit(limit)
  .lean();
```

\$meta: 'textScore' expone el puntaje de relevancia que calcula el índice. Sin él, los resultados vuelven en orden arbitrario y el buscador se siente roto. Es el detalle que separa un buscador usable de un **find** con expresión regular.

Seed — Persona B (estructura) · Persona A y B (contenido)

src/seeds/glossary.js

```
import { GlossaryTerm } from '../models/index.js';

// moduleOrder: 1-6, o null si el término es transversal.
// Persona A → módulos 1 a 4 · Persona B → módulos 5 y 6
export const GLOSSARY = [
  {
    term: 'Fondo de emergencia',
    slug: 'fondo-de-emergencia',
    definition: 'Dinero apartado y disponible que cubre entre 3 y 6 meses de tus gastos. Sirve para imprevistos –una urgencia médica, quedarte sin trabajo– sin tener que endeudarte.',
    keywords: ['fondo de emergencia', 'colchón', 'ahorro de emergencia', 'imprevistos'],
    moduleOrder: 1,
    displayOrder: 1,
  },
  {
    term: 'Regla 50/30/20',
    slug: 'regla-50-30-20',
    definition: 'Forma simple de repartir tus ingresos: 50 % para necesidades, 30 % para gustos y 20 % para ahorro o pago de deudas.',
    keywords: ['regla 50 30 20', 'reparto de ingresos', 'presupuesto'],
    moduleOrder: 2,
    displayOrder: 2,
  },
  {
    term: 'Método avalancha',
    slug: 'metodo-avalancha',
    definition: 'Estrategia para salir de deudas pagando primero la de mayor tasa de interés. Es la que menos intereses te hace pagar en total.',
    keywords: ['metodo avalancha', 'avalancha', 'pagar deudas', 'tasa mas alta'],
    moduleOrder: 3,
    displayOrder: 3,
  },
  // ... Persona A completa hasta el módulo 4 · Persona B agrega módulos 5 y 6
];

export const seedGlossary = async () => {
  for (const t of GLOSSARY) {
    await GlossaryTerm.updateOne({ slug: t.slug }, { $set: t }, { upsert: true, runValidators: true });
  }
  console.log(`✅ ${GLOSSARY.length} términos de glosario cargados`);
};
```


Colección: `modules` (campo agregado: `theoryContent`). No hay colección nueva — se decidió no crear `lessons` separada porque la relación módulo↔teoría es 1 a 1 y agregar una colección solo para eso introduce un join innecesario.

Modelo `Module` actualizado (reemplaza al de la Parte 2 — F03, agrega los dos campos de teoría):

Ver el modelo `Module` completo en la Parte 2 — F03, que ya incluye `theoryContent` y `estimatedReadingMinutes`. No se crea ninguna colección nueva.

F12 — Integración Google Forms · Colección `testResults`

Se implementa el modelo desde ahora aunque los endpoints sean opcionales. Cuesta cinco minutos y deja la puerta abierta.

`src/models/TestResult.js`

```
import mongoose from 'mongoose';
const { Schema } = mongoose;

const testAnswerSchema = new Schema({
  questionNumber: { type: Number, required: true, min: 1 },
  conceptTag: { type: String, required: true, lowercase: true },
  isCorrect: { type: Boolean, required: true },
}, { _id: false });

const testResultSchema = new Schema({
  studentCode: { type: String, required: true, uppercase: true, trim: true },
  userId: { type: Schema.Types.ObjectId, ref: 'User', default: null },
  classCode: { type: String, required: true, uppercase: true, trim: true },
  testType: { type: String, required: true, enum: ['pretest', 'posttest'] },
  score: { type: Number, required: true, min: 0 },
  totalQuestions: { type: Number, required: true, min: 1 },
  scorePercentage: { type: Number, required: true, min: 0, max: 100 },
  answers: { type: [testAnswerSchema], default: [] },
  submittedAt: { type: Date, required: true },
  importedAt: { type: Date, default: Date.now },
  source: { type: String, default: 'google_forms' },
}, { timestamps: true });

testResultSchema.index({ studentCode: 1, testType: 1 }, { unique: true });
testResultSchema.index({ classCode: 1, testType: 1 });

testResultSchema.pre('validate', function (next) {
  if (this.score > this.totalQuestions) {
    return next(new Error('El puntaje no puede superar el total de preguntas.'));
  }
}
```

```

    this.scorePercentage = Math.round((this.score / this.totalQuestions) *
10000) / 100;
    next();
  });

export default mongoose.model('TestResult', testResultSchema);

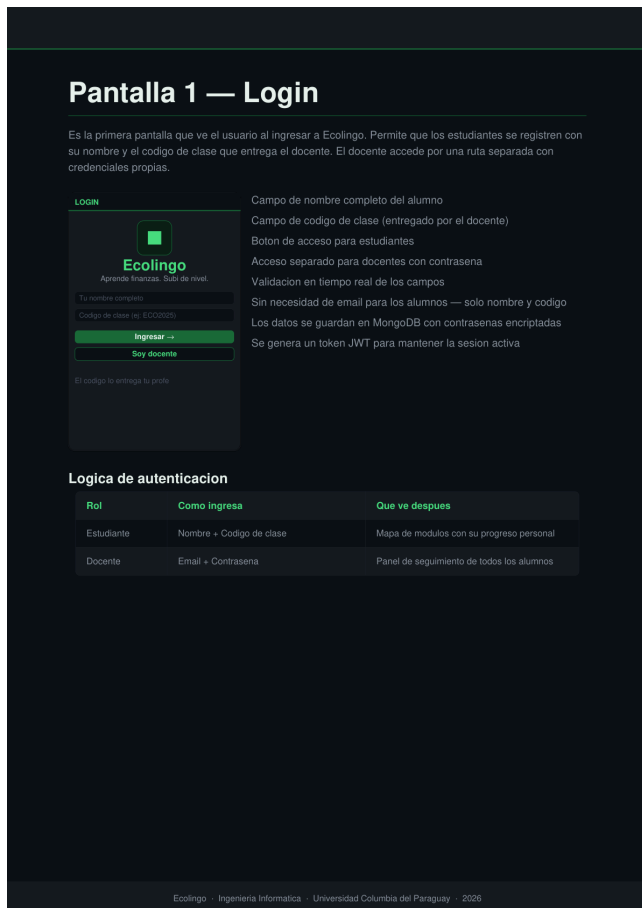
```

PARTE 3 — Frontend por feature

Para cada feature: la pantalla del Documento de Diseño, las rutas, los endpoints que consume, y el detalle funcional — qué incluye, qué no, sus criterios de aceptación y los riesgos concretos de construirla. Es el mismo contenido que tenía [02-FEATURES.md](#), con las pantallas ya incrustadas como imagen.

F01 — Autenticación

Dueño	Ambos, en pareja
Endpoints	2, 3, 4, 26
Colecciones	users (L/E), classes (L)
Pantalla	Diseño p. 4
Rutas	/login
Estimación	12–16 h
Origen	RF1, RF2, D-05 revisada



Objetivo

Permitir que un alumno entre con su nombre, código de alumno, código de clase y un **PIN de 4 dígitos**, y que el docente entre con email y contraseña. Emitir el JWT que sostiene toda la sesión.

Incluye

- Pantalla de login con los dos modos: alumno y docente
- Campo de PIN (4 dígitos, teclado numérico en móvil) además de nombre, código de alumno y código de clase
- Alta automática del alumno en su primer ingreso, fijando el PIN que envía
- Login de docente con verificación bcrypt
- Emisión y verificación del JWT
- Middlewares `requireAuth` y `requireRole`
- `AuthContext` en el frontend, persistencia en `localStorage`
- Redirección por rol: alumno a `/mapa`, docente a `/docente`
- Rehidratación de sesión al recargar

No incluye

- Recuperación de PIN por autoservicio — no se implementa. Si el alumno lo olvida, lo resuelve el docente desde el panel (endpoint 26, ver F08).
- Recuperación de contraseña del docente — no se implementa. Con un solo docente, se resuelve por seed.
- Cierre de sesión — va en F02, dentro del shell.
- Edición de perfil — va en F09.

Criterios de aceptación

- ☐ Un alumno nuevo ingresa con nombre + código de alumno + código de clase + PIN y queda registrado, con ese PIN fijado
- ☐ El mismo alumno vuelve a entrar con los mismos datos y el mismo PIN, y recupera su progreso
- ☐ El mismo alumno con el PIN incorrecto recibe **INVALID_PIN**, sin importar que el nombre y el código sean correctos
- ☐ Un código de alumno ya usado con **otro nombre** devuelve **STUDENT_CODE_TAKEN**
- ☐ Un código de clase inexistente o inactivo devuelve **INVALID_CLASS_CODE**
- ☐ Un PIN que no son 4 dígitos numéricos devuelve **VALIDATION_ERROR**
- ☐ El docente ingresa con email y contraseña y llega a **/docente**
- ☐ Credenciales de docente incorrectas devuelven **INVALID_CREDENTIALS**, con el mismo mensaje para email inexistente y contraseña errada
- ☐ El JWT contiene exactamente los claims de la Parte 0.2 — ni uno más
- ☐ **passwordHash** y **pinHash** no aparecen en ninguna respuesta de la API
- ☐ Un token expirado o inválido redirige a **/login** y limpia el almacenamiento
- ☐ Al recargar la página, la sesión se mantiene
- ☐ Funciona en 375 px

Dependencias

Base de datos completa. Modelos **User** y **Class** con sus índices, seed del docente y de la clase **EC02025**.

Riesgos

El índice **{ email: 1 }** necesita **sparse: true**. Sin eso, todos los alumnos tienen **email: null** y el índice único los considera duplicados.

No revelar qué emails existen ni si el error fue de nombre o de PIN por separado más allá de lo que ya distingue el contrato. **INVALID_CREDENTIALS** con idéntico mensaje en ambos casos para el docente.

El PIN es de 4 dígitos — entropía baja a propósito. Es apropiado para el contexto (aula, adolescentes, sin nada más valioso que XP y progreso detrás). No es una contraseña de producción bancaria; documentarlo así si el tribunal pregunta.

Normalización consistente. **fullName** a *Title Case* con espacios colapsados; **studentCode** y **classCode** a mayúsculas.

Rehidratación sin parpadeo. Al montar la app, mientras se verifica el token contra **GET /auth/me**, mostrar un estado de carga.

F02 — Shell de la aplicación

Dueño	Ambos, en pareja
-------	------------------

Endpoints	—
Colecciones	—
Pantalla	Diseño p. 5 (tabs), p. 13 (rutas)
Rutas	Todas
Estimación	6–8 h
Origen	Diseño

Flujo de Navegacion entre Pantallas Este diagrama muestra como se conectan todas las pantallas de Ecolingo y en que condiciones el usuario pasa de una a otra. <table><tr><th>Desde</th><th>Accion del usuario</th><th>Va a</th></tr><tr><td>Login</td><td>Ingresar con nombre + código (alumno)</td><td>Mapa de Módulos</td></tr><tr><td>Login</td><td>Ingresar con email + contraseña (docente)</td><td>Panel del Docente</td></tr><tr><td>Mapa de Módulos</td><td>Clickear modulo activo (no bloqueado)</td><td>Leccion (Pregunta)</td></tr><tr><td>Mapa de Módulos</td><td>Tab "Ranking"</td><td>Ranking de Clase</td></tr><tr><td>Mapa de Módulos</td><td>Tab "Logros"</td><td>Logros e Insignias</td></tr><tr><td>Leccion</td><td>Responder las 3 situaciones</td><td>Resultado del Modulo</td></tr><tr><td>Leccion</td><td>Clickear X (cerrar)</td><td>Mapa de Módulos (progreso guardado)</td></tr><tr><td>Resultado</td><td>Boton "Ver mi progreso"</td><td>Mapa de Módulos</td></tr><tr><td>Resultado</td><td>Boton "Volver al mapa"</td><td>Mapa de Módulos</td></tr><tr><td>Ranking</td><td>Tab "Módulos"</td><td>Mapa de Módulos</td></tr><tr><td>Logros</td><td>Tab "Módulos"</td><td>Mapa de Módulos</td></tr><tr><td>Panel Docente</td><td>Boton "Activar pre-test"</td><td>Alumnos ven el test habilitado</td></tr><tr><td>Panel Docente</td><td>Boton "Exportar CSV"</td><td>Descarga archivo de datos</td></tr></table> Rutas de la aplicacion <table><tr><th>Ruta URL</th><th>Pantalla</th><th>Acceso</th></tr><tr><td>/login</td><td>Login</td><td>Publica (sin autentificacion)</td></tr><tr><td>/mapa</td><td>Mapa de Módulos</td><td>Solo alumnos autenticados</td></tr><tr><td>/leccion/:moduleId</td><td>Leccion</td><td>Solo alumnos autenticados</td></tr><tr><td>/resultado/:moduleId</td><td>Resultado</td><td>Solo alumnos autenticados</td></tr><tr><td>/docente</td><td>Panel del Docente</td><td>Solo docentes autenticados</td></tr><tr><td>/docente/clase/:claseId</td><td>Detalle de clase</td><td>Solo docentes autenticados</td></tr><tr><td>/pre-test</td><td>Pre-test</td><td>Alumnos (cuando docente lo habilite)</td></tr></table>			Desde	Accion del usuario	Va a	Login	Ingresar con nombre + código (alumno)	Mapa de Módulos	Login	Ingresar con email + contraseña (docente)	Panel del Docente	Mapa de Módulos	Clickear modulo activo (no bloqueado)	Leccion (Pregunta)	Mapa de Módulos	Tab "Ranking"	Ranking de Clase	Mapa de Módulos	Tab "Logros"	Logros e Insignias	Leccion	Responder las 3 situaciones	Resultado del Modulo	Leccion	Clickear X (cerrar)	Mapa de Módulos (progreso guardado)	Resultado	Boton "Ver mi progreso"	Mapa de Módulos	Resultado	Boton "Volver al mapa"	Mapa de Módulos	Ranking	Tab "Módulos"	Mapa de Módulos	Logros	Tab "Módulos"	Mapa de Módulos	Panel Docente	Boton "Activar pre-test"	Alumnos ven el test habilitado	Panel Docente	Boton "Exportar CSV"	Descarga archivo de datos	Ruta URL	Pantalla	Acceso	/login	Login	Publica (sin autentificacion)	/mapa	Mapa de Módulos	Solo alumnos autenticados	/leccion/:moduleId	Leccion	Solo alumnos autenticados	/resultado/:moduleId	Resultado	Solo alumnos autenticados	/docente	Panel del Docente	Solo docentes autenticados	/docente/clase/:claseId	Detalle de clase	Solo docentes autenticados	/pre-test	Pre-test	Alumnos (cuando docente lo habilite)
Desde	Accion del usuario	Va a																																																																		
Login	Ingresar con nombre + código (alumno)	Mapa de Módulos																																																																		
Login	Ingresar con email + contraseña (docente)	Panel del Docente																																																																		
Mapa de Módulos	Clickear modulo activo (no bloqueado)	Leccion (Pregunta)																																																																		
Mapa de Módulos	Tab "Ranking"	Ranking de Clase																																																																		
Mapa de Módulos	Tab "Logros"	Logros e Insignias																																																																		
Leccion	Responder las 3 situaciones	Resultado del Modulo																																																																		
Leccion	Clickear X (cerrar)	Mapa de Módulos (progreso guardado)																																																																		
Resultado	Boton "Ver mi progreso"	Mapa de Módulos																																																																		
Resultado	Boton "Volver al mapa"	Mapa de Módulos																																																																		
Ranking	Tab "Módulos"	Mapa de Módulos																																																																		
Logros	Tab "Módulos"	Mapa de Módulos																																																																		
Panel Docente	Boton "Activar pre-test"	Alumnos ven el test habilitado																																																																		
Panel Docente	Boton "Exportar CSV"	Descarga archivo de datos																																																																		
Ruta URL	Pantalla	Acceso																																																																		
/login	Login	Publica (sin autentificacion)																																																																		
/mapa	Mapa de Módulos	Solo alumnos autenticados																																																																		
/leccion/:moduleId	Leccion	Solo alumnos autenticados																																																																		
/resultado/:moduleId	Resultado	Solo alumnos autenticados																																																																		
/docente	Panel del Docente	Solo docentes autenticados																																																																		
/docente/clase/:claseId	Detalle de clase	Solo docentes autenticados																																																																		
/pre-test	Pre-test	Alumnos (cuando docente lo habilite)																																																																		
Ecolingo · Ingeniería Informática · Universidad Columbia del Paraguay · 2026																																																																				

Objetivo

El almacén que sostiene todas las pantallas: enrutado protegido, navegación por tabs, cabecera con XP y racha, y los estados compartidos de carga, error y vacío.

Incluye

- Todas las rutas del Contrato §5.3, con sus guards por rol
- Layout del alumno: cabecera + tabs Módulos / Ranking / Logros
- Cabecera con nombre, XP total y racha (Diseño p. 5)
- Layout del docente
- Componentes comunes: **Button**, **Card**, **ProgressBar**, **Spinner**, **EmptyState**, **ErrorState**, **Modal**
- Estado de carga y de error, unificados
- Página 404 y redirección por defecto
- Cierre de sesión

- Comportamiento responsivo base

No incluye

- El contenido de cada pantalla — cada feature pone el suyo
- Datos reales en la cabecera: se consume `GET /auth/me`, que ya existe

Criterios de aceptación

- ☐ Un alumno no autenticado que entra a `/mapa` va a parar a `/login`
- ☐ Un alumno que entra a `/docente` recibe rechazo por rol
- ☐ Un docente que entra a `/mapa` recibe rechazo por rol
- ☐ Los tabs navegan entre Módulos, Ranking y Logros sin recargar
- ☐ El tab activo se distingue visualmente
- ☐ La cabecera muestra nombre, XP y racha reales
- ☐ Cerrar sesión limpia el token y lleva a `/login`
- ☐ Una ruta inexistente muestra 404 con salida al mapa
- ☐ `Spinner`, `EmptyState` y `ErrorState` existen y son reutilizables
- ☐ Ningún componente usa colores hexadecimales literales
- ☐ Layout correcto en 375 px, 768 px y 1280 px

Dependencias

F01.

Riesgos

Es zona compartida. Todo lo de esta feature lo van a tocar los dos después. Definir bien las props de los componentes comunes acá evita que cada uno haga su propio botón.

La cabecera se desactualiza. Muestra XP y racha, que cambian al terminar una lección. Hace falta un mecanismo para refrescarla desde otra feature — lo más simple es exponer un `refreshUser()` en el `AuthContext` que F05 invoque. **Acordarlo en esta feature**, porque si no F05 termina duplicando estado.

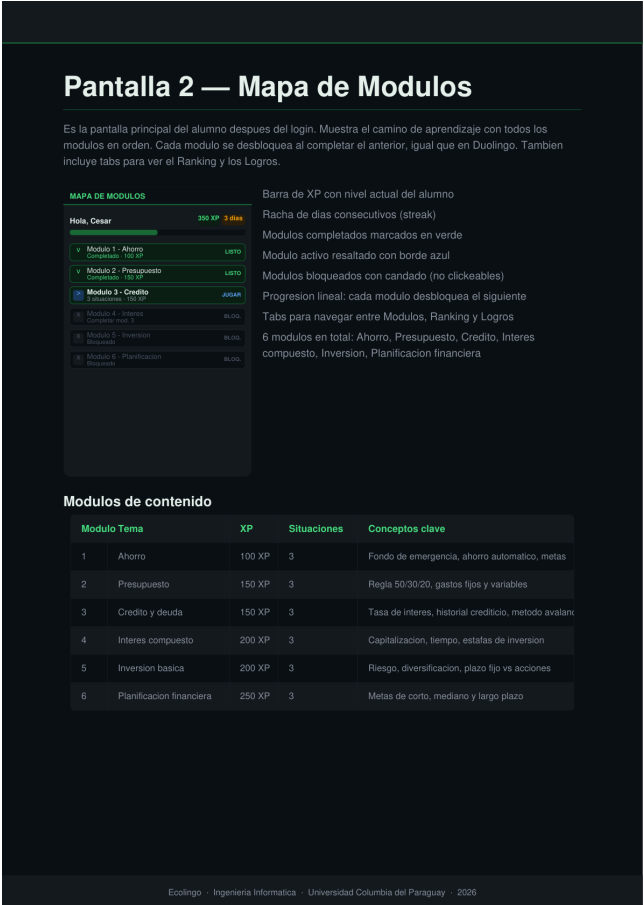
Tres clics máximo. La tesis §4.5 compromete que llegar a cualquier sección requiera como máximo tres clics. Los tabs lo resuelven, pero conviene verificarlo explícitamente con Perfil y Buscador incluidos.

Móvil primero. Es probable que los alumnos entren desde el celular. Los tabs en 375 px tienen que ser cómodos con el pulgar, no una fila apretada.

F03 — Mapa de Módulos

Dueño	Persona A
Endpoints	5, 6, 12
Colecciones	<code>modules</code> (L), <code>progress</code> (L/E)

Pantalla	Diseño p. 5
Rutas	/mapa
Estimación	8–12 h
Origen	RF3



Nota sobre F11. El botón "Jugar" de esta pantalla ahora navega a [/leccion/:moduleId/teoria](#) (F11) en lugar de crear el intento directamente. El resto de esta feature no cambia.

Objetivo

Pantalla principal del alumno. Muestra el camino de aprendizaje con los seis módulos, su estado de desbloqueo y el progreso personal.

Incluye

- Endpoint 5 devolviendo módulos + progreso + resumen en una sola llamada
- Endpoint 6 con el detalle de un módulo
- Lógica de desbloqueo lineal, calculada **en el servidor**
- Creación perezosa de los documentos de **progress** en el primer acceso
- Renderizado de los tres estados: completado, disponible, bloqueado
- Barra de XP y racha
- Botón "Jugar" que navega a [/leccion/:moduleId/teoria](#) (F11), o "Continuar" si hay intento activo
- Enlace a Ranking y Logros vía tabs

No incluye

- El contenido teórico — F11
- El motor de preguntas — F04
- Ranking y Logros — F06 y F07

Criterios de aceptación

- ☐ Un alumno nuevo ve el módulo 1 disponible y los otros cinco bloqueados
- ☐ Al completar el módulo N, el N+1 pasa a disponible sin recargar manualmente
- ☐ Los bloqueados no son clickeables y muestran su requisito en texto
- ☐ El activo se resalta con borde **info**
- ☐ Los completados se marcan en verde con su XP obtenido
- ☐ El array llega ordenado por **order** y el frontend no reordena
- ☐ Con **activeAttemptId** presente, el botón dice "Continuar" y lleva directo al intento en curso (saltando la teoría)
- ☐ El estado lo calcula el servidor; el cliente no infiere desbloques
- ☐ Estados de carga, error y reintento resueltos
- ☐ Los seis módulos se ven bien en 375 px

Dependencias

F01, F02. Seed de módulos cargado, con **theoryContent**.

Riesgos

Nunca calcular el desbloqueo en el cliente.

Creación perezosa de **progress.** Se crea al primer **GET /modules**, no al registrarse.

activeAttemptId es la clave de la reanudación. Sin ese campo, un alumno que cierra la lección a la mitad no puede volver.

Cache al volver de la lección. Al terminar un módulo y volver al mapa, los datos tienen que estar frescos.

F04 — Motor de Lección

Dueño	Persona B
Endpoints	7, 8, 9, 10
Colecciones	questions (L), moduleAttempts (L/E), progress (E), users (E), userBadges (E)
Pantalla	Diseño p. 6
Rutas	/leccion/:attemptId
Estimación	16–22 h

Origen

RF4

Pantalla 3 — Leccion (Motor de Preguntas)

Es el corazon de la gamificacion. El alumno responde situaciones practicas de economia personal con 3 opciones de respuesta. Cuenta con vidas (como Duolingo), barra de progreso, retroalimentacion inmediata con explicacion y acumulacion de XP por cada respuesta correcta.

LECCION — PREGUNTA

Situacion 2 de 3

Situacion
Tienes una deuda en tarjeta al 20% y un prestamo al 18%. ¿Por cual pagas primero?

Por la tarjeta de credito (m...)

Por el prestamo personal

Por la de cuota mas baja

¡Correcto! +50 XP
El metodo de abanico reduce el total de...

Siguiente →

Barra de progreso (situacion X de 3)

Sistema de 3 vidas — se pierde una por respuesta incorrecta

Tarjeta de situacion con contexto real y cercano al alumno

Tres opciones de respuesta (A, B, C)

Feedback inmediato: verde si es correcto, rojo si no

Explicacion de por que la respuesta es correcta o incorrecta

+50 XP por cada respuesta correcta

Boton "Siguiente" solo aparece despues de responder

La opcion correcta siempre se muestra aunque la respuesta sea erronea

Mecanica de vidas y XP

Elemento	Descripcion	Impacto
Vidas (Corazones)	Cada alumno empieza con 3 vidas por leccion	Incentiva prestar atencion antes de res
XP por correcta	+50 XP por cada respuesta acertada	Motiva completar con el mayor puntaje
Feedback inmediato	Explicacion pedagogica despues de cada respuesta	Refuerza el aprendizaje del concepto
Progreso visible	Barra que avanza con cada situacion	El alumno sabe cuanto falta para termi

Ecolingo · Ingeniería Informática · Universidad Columbia del Paraguay · 2026

Objetivo

El corazón de la gamificación. El alumno responde situaciones prácticas con tres opciones, pierde vidas al error, gana XP al acertar y recibe explicación inmediata en ambos casos.

Es la feature más compleja del sistema y la que sostiene la hipótesis del TCC.

Incluye

- Iniciar intento (7), consultar estado (8), responder (9), abandonar (10)
- Sistema de tres vidas
- Corrección **en el servidor** y devolución de explicación
- Acreditación de XP con la regla anti-farming (D-08)
- Actualización de **progress** al cerrar el intento
- Actualización de racha
- Evaluación de insignias
- Cálculo de **durationSeconds**
- Pantalla: barra de progreso, corazones, tarjeta de situación, tres opciones, feedback, botón siguiente
- Reanudación tras recarga
- Cierre con X

No incluye

- La pantalla de resultado — F05, aunque este motor produce sus datos
- Catálogo de insignias — F07, aunque acá se disparan
- Contenido teórico — F11, sin resolver

Criterios de aceptación

- ☐ Iniciar genera el intento y devuelve la primera situación
- ☐ `correctOptionKey` y `explanation` **no viajan** al cliente antes de responder — verificado en la pestaña de red
- ☐ Una respuesta correcta suma 50 XP y avanza
- ☐ Una incorrecta descuenta una vida y **también** muestra la explicación
- ☐ Al llegar a cero vidas el intento pasa a `failed` y se cierra
- ☐ Responder todas con vidas restantes deja el intento en `completed`
- ☐ El XP se acredita **solo en la primera finalización**; repetir da 0
- ☐ Un segundo intento simultáneo del mismo módulo devuelve `ATTEMPT_ALREADY_ACTIVE`
- ☐ Reenviar la misma respuesta devuelve `QUESTION_ALREADY_ANSWERED`
- ☐ Recargar en medio de la lección restaura el estado exacto
- ☐ Cerrar con X marca `abandoned` sin otorgar XP
- ☐ Al finalizar se actualizan `progress`, `totalXp`, `weeklyXp` y la racha
- ☐ Las insignias correspondientes quedan desbloqueadas
- ☐ Feedback en verde o rojo según el diseño p. 6
- ☐ El botón siguiente aparece solo después de responder
- ☐ Funciona en 375 px

Dependencias

F01, F02. Seed de preguntas cargado. `progress` creado por F03 o al registrarse.

Riesgos

Filtrar la respuesta correcta invalida el piloto. Usar siempre la proyección pública del modelo `Question`. Verificarlo mirando la respuesta real en la pestaña de red, no confiando en el código.

Doble clic. Un alumno impaciente toca dos veces. Sin protección, se registran dos respuestas y el XP se duplica. El índice único parcial cubre la creación de intentos; para las respuestas hay que validar contra `currentQuestionOrder` y deshabilitar el botón mientras la petición está en vuelo.

El XP se calcula solo en el servidor. El cliente muestra lo que le devuelven. Si el frontend suma por su cuenta, aparecen discrepancias imposibles de depurar entre lo que ve el alumno y lo que hay en la base.

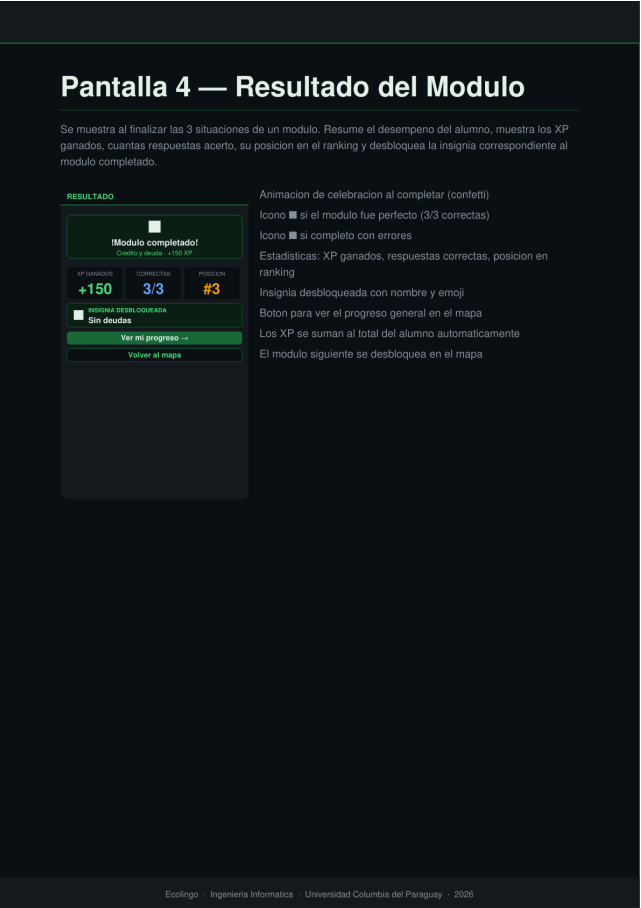
El orden de las escrituras finales importa. Al cerrar un intento hay que actualizar `moduleAttempts`, `progress`, `users` y `userBadges`. Definí una secuencia fija y respetala; si se hace en distinto orden según el camino, aparecen estados inconsistentes muy difíciles de reproducir.

Idempotencia de las insignias. El servicio se ejecuta al final de cada intento y reevalúa todo. El índice único de `userBadges` es lo que impide duplicados: hay que capturar el error 11000 e ignorarlo, no evitar la inserción con una lectura previa.

Conexión intermitente. Si el alumno pierde conexión al responder, tiene que poder reintentar sin perder el intento. El endpoint 8 existe para eso.

F05 — Resultado del Módulo

Dueño	Persona A
Endpoints	11
Colecciones	<code>moduleAttempts</code> (L), <code>progress</code> (L), <code>userBadges</code> (L), <code>users</code> (L)
Pantalla	Diseño p. 7
Rutas	<code>/resultado/:attemptId</code>
Estimación	6–9 h
Origen	RF4, RF6



Objetivo

Cierre de la experiencia de un módulo: XP ganado, aciertos, posición en el ranking, insignias desbloqueadas y confirmación de que el siguiente módulo se abrió.

Incluye

- Endpoint 11, idempotente y de solo lectura
- Tarjetas de estadísticas: XP, correctas, posición
- Insignias desbloqueadas con nombre y emoji

- Animación de celebración con Framer Motion
- Distinción visual entre módulo perfecto y con errores
- Botones "Ver mi progreso" y "Volver al mapa"
- Refresco de la cabecera del shell
- Manejo del caso `isFirstCompletion: false`

No incluye

- El motor que produce los datos — F04
- El catálogo completo de insignias — F07

Criterios de aceptación

- ☐ Al terminar una lección se llega automáticamente a esta pantalla
- ☐ Muestra XP ganado, aciertos sobre total y posición en el ranking
- ☐ Un módulo perfecto se distingue visualmente de uno con errores
- ☐ Las insignias nuevas aparecen con nombre y emoji; si no hubo, la sección no se muestra
- ☐ La animación de celebración se dispara al montar
- ☐ En una repetición muestra `xpEarned: 0` y **atenúa la celebración**
- ☐ Ambos botones vuelven al mapa
- ☐ El siguiente módulo aparece desbloqueado al volver
- ☐ La cabecera refleja el XP actualizado
- ☐ Recargar la pantalla muestra lo mismo — es idempotente
- ☐ Un intento de otro alumno devuelve `NOT_RESOURCE_OWNER`
- ☐ Funciona en 375 px

Dependencias

F04 debe estar terminada. Es la única dependencia que cruza entre personas. Persona A puede construir toda la pantalla contra un *mock* con el shape del endpoint 11 —que ya está fijado en el contrato — y sustituirlo cuando B termine.

Riesgos

Es el punto de coordinación entre A y B. El contrato ya fija el shape exacto, así que no hay que acordar nada nuevo. Lo que sí conviene: que A arme el *mock* copiando literalmente el ejemplo del contrato, no una versión propia.

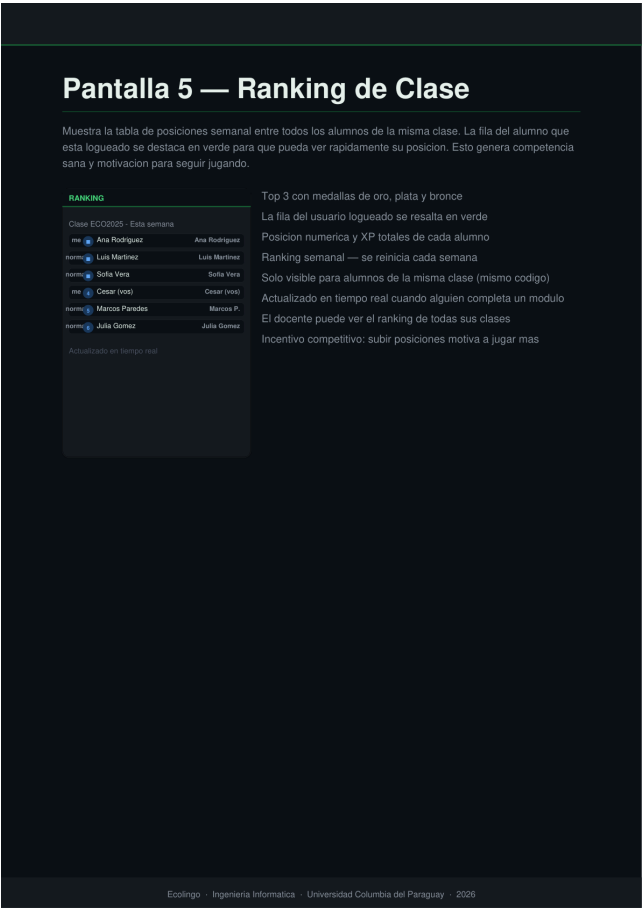
Repetir un módulo no debe celebrarse igual. Si un alumno rehace el módulo 1 y ve la misma fiesta con "+0 XP", la interfaz se siente rota. `isFirstCompletion` existe para eso.

Intento fallido. Si el alumno perdió las tres vidas, el intento es **failed**. Esta pantalla tiene que manejarlo con un tono distinto: sin confeti, con invitación a reintentar. **No está en el mockup del diseño** — hay que resolverlo respetando el sistema visual.

La celebración no puede tapar la información. El confeti es agradable la primera vez y molesto la quinta. Que sea breve y no bloquee los botones.

F06 — Ranking de Clase

Dueño	Persona A
Endpoints	15
Colecciones	users (L)
Pantalla	Diseño p. 8
Rutas	/ranking
Estimación	5–7 h
Origen	RF6



Objetivo

Tabla de posiciones semanal entre los alumnos de la misma clase, con la fila propia resaltada.

Incluye

- Endpoint 15 con **scope** semanal e histórico
- Reinicio semanal perezoso (D-09)
- Desempate estable
- Medallas para el top 3
- Fila propia resaltada en verde
- Bloque **currentUser** aunque quede fuera del límite

- Estado vacío cuando es la única persona con XP

No incluye

- Ranking entre clases distintas
- Actualización en vivo por websocket — se refresca al entrar

Criterios de aceptación

- ☐ Solo aparecen alumnos de la misma clase, activos
- ☐ Ordenado por `weeklyXp` descendente
- ☐ El top 3 lleva medallas de oro, plata y bronce
- ☐ La fila propia está resaltada y `isCurrentUser` viene del servidor
- ☐ `currentUser` está presente aunque el alumno esté en el puesto 15
- ☐ A igual XP, gana quien llegó primero, y el orden es estable entre recargas
- ☐ Al completar un módulo, la posición se actualiza al volver a entrar
- ☐ Con `scope=alltime` ordena por `totalXp`
- ☐ La semana va de lunes a domingo en hora de Paraguay
- ☐ Estado vacío resuelto
- ☐ Funciona en 375 px

Dependencias

F01, F02.

Riesgos

El reinicio semanal no usa cron. El plan gratuito de Render suspende las instancias, así que un cron no correría de forma confiable. Se evalúa en cada lectura o escritura de XP (D-09).

Desempate inestable. Sin criterio de desempate, dos alumnos con el mismo XP intercambian posiciones entre recargas. Se ve como un bug y desmotiva.

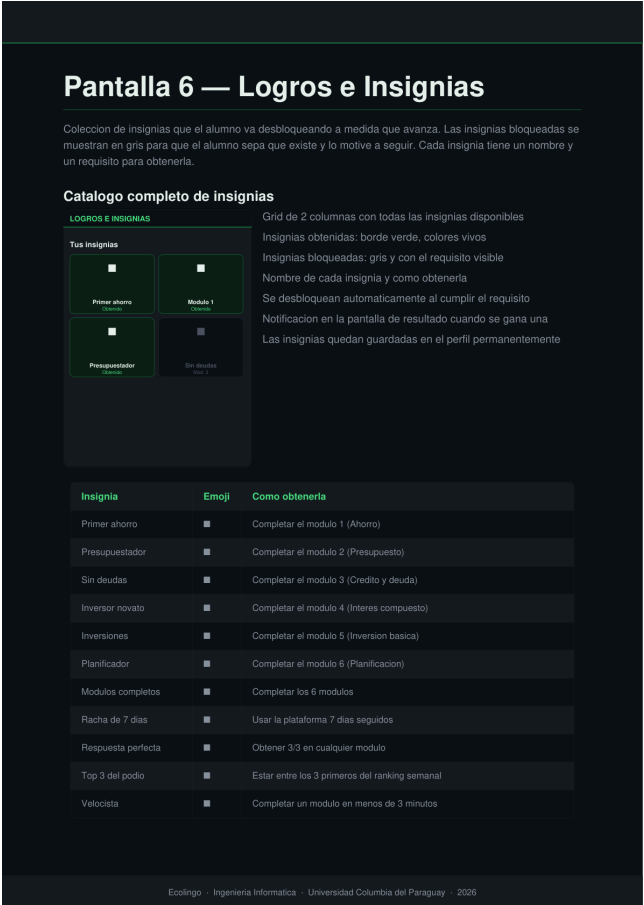
El diseño dice "actualizado en tiempo real" pero no lo es. Se refresca al entrar a la pantalla. Es suficiente para el piloto y conviene no prometer más en la defensa.

Privacidad. El ranking muestra nombres completos de menores a sus compañeros de clase. Es lo mismo que una lista de notas en cartelera, pero **nunca debe exponer** `studentCode` — ese campo solo aparece en endpoints de docente.

F07 — Logros e Insignias

Dueño	Persona A
Endpoints	13, 14
Colecciones	<code>badges</code> (L), <code>userBadges</code> (L)

Pantalla	Diseño p. 9
Rutas	/logros
Estimación	4–6 h
Origen	RF6



Objetivo

Colección de las once insignias, mostrando las obtenidas a color y las pendientes en gris con su requisito visible.

Incluye

- Endpoints 13 y 14
- Grid con **las once** insignias, obtenidas y no obtenidas
- Obtenidas: borde verde, emoji a color, fecha
- Bloqueadas: gris, con el requisito legible
- Contador de progreso
- Orden por `displayOrder`

No incluye

- La evaluación de criterios — vive en F04
- Notificación de insignia nueva — se muestra en F05

Criterios de aceptación

- ☐ Aparecen las once insignias, sin filtrar
- ☐ Las obtenidas se distinguen claramente de las bloqueadas
- ☐ Cada bloqueada muestra cómo conseguirla
- ☐ Ordenadas por `displayOrder`, sin reordenar en el cliente
- ☐ El contador muestra obtenidas sobre total
- ☐ Al desbloquear una, aparece a color en la siguiente visita
- ☐ Grid de 2 columnas en móvil, más en pantallas grandes
- ☐ Estado de carga y error resueltos

Dependencias

F01, F02. Seed de insignias. F04 para que se desbloqueen de verdad — pero la pantalla se puede construir y probar antes.

Riesgos

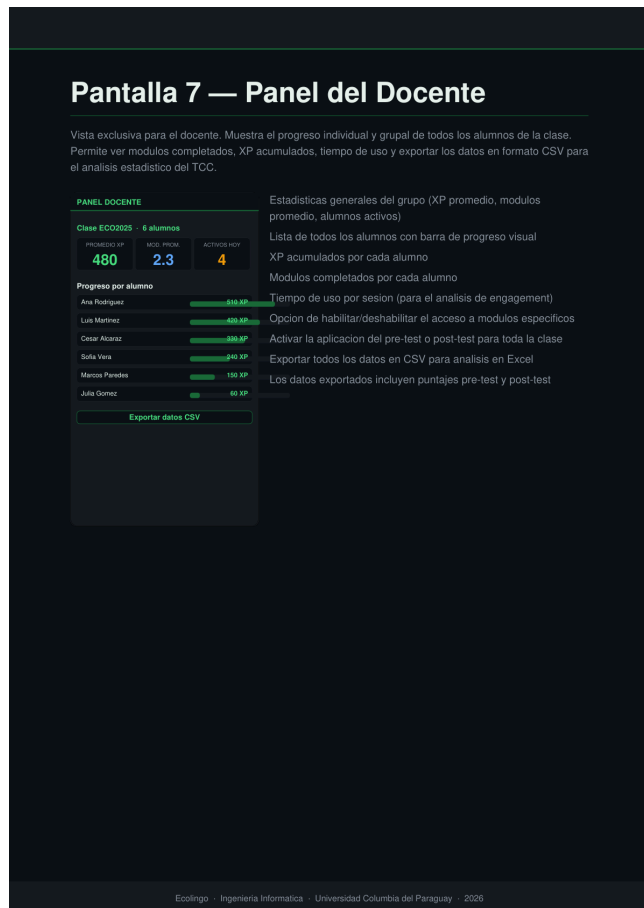
Mostrar las bloqueadas es el punto de la pantalla. Es lo que genera la motivación. Filtrarlas para que "se vea más limpio" vacía la feature de sentido.

Los emojis dependen de la fuente del sistema. En algunos Android viejos pueden no renderizar. Vale la pena probarlo en un celular real antes del piloto.

Dos insignias son difíciles de verificar manualmente: `RACHA_7_DIAS` requiere siete días, y `TOP_3_PODIO` depende del ranking. Para probarlas, modificá los datos en `ecolingo_dev`, nunca en producción.

F08 — Panel del Docente

Dueño	Persona B
Endpoints	16, 17, 18, 19, 20
Colecciones	todas (lectura), <code>classes</code> (E)
Pantalla	Diseño p. 10
Rutas	<code>/docente</code> , <code>/docente/clase/:classCode</code>
Estimación	14–20 h
Origen	RF7



Incluye la UI del endpoint 26 (reseteo de PIN): un botón por alumno en la lista, que muestra el PIN nuevo en un modal única vez, con instrucción de comunicarlo verbalmente.

Objetivo

Vista de seguimiento para el docente: progreso individual y grupal, estadísticas de la clase, y exportación de datos para el análisis del TCC.

Incluye

- Endpoints 16 a 20
- Tarjetas de estadísticas: XP promedio, módulos promedio, activos hoy
- Lista de alumnos con barra de progreso, ordenable
- Desglose por módulo con dificultad y tiempo promedio
- **Gráficos con Recharts** (decisión T-03)
- Habilitar y deshabilitar pre-test y post-test
- Exportación CSV en tres niveles de detalle
- Guard de rol docente

No incluye

- Edición de datos de alumnos
- Creación de clases — por seed
- Resultados de los tests — F12

Criterios de aceptación

- ☐ Solo accesible con rol docente; un alumno recibe `FORBIDDEN_ROLE`
- ☐ Un docente no puede ver una clase de otro docente
- ☐ Las estadísticas coinciden con los datos reales de la base
- ☐ La lista de alumnos se ordena por los cuatro criterios
- ☐ `studentCode` visible solo acá
- ☐ Al menos dos gráficos con Recharts, con la paleta del diseño
- ☐ Habilitar el pre-test se refleja en la vista del alumno
- ☐ El CSV descarga con las columnas especificadas
- ☐ **El CSV abre bien en Excel con tildes y ñ** — UTF-8 con BOM
- ☐ `include=answers` exporta el detalle por respuesta
- ☐ Estado vacío cuando la clase no tiene alumnos
- ☐ Usable en tablet

Dependencias

F01, F02. Datos reales de alumnos para que las estadísticas signifiquen algo — se puede construir con datos de prueba en `dev`.

Riesgos

El BOM del CSV no es un detalle. Sin él, Excel en Windows muestra "Rodríguez" corrupto. Ese archivo es un anexo del TCC.

Las agregaciones pueden salir caras. Calcular promedios por módulo cruzando `moduleAttempts` requiere pipelines de agregación. Con 20 alumnos no hay problema de rendimiento, pero conviene escribirlas bien porque son un buen material de defensa.

El panel es la herramienta del docente durante el piloto. Si falla, el docente pierde visibilidad de sus alumnos. Es la feature que más conviene probar con datos realistas antes de septiembre.

Los gráficos deben usar la paleta. Recharts trae colores por defecto que rompen el sistema visual. Hay que pasarle los tokens explícitamente.

Alcance. Es la feature más grande junto con F04 y está marcada como "extra" en el brief. Si el tiempo aprieta, priorizá estadísticas y exportación CSV; los gráficos son lo primero que se recorta.

F09 — Perfil de usuario

Dueño	Persona A
Endpoints	23, 24
Colecciones	<code>users</code> (L/E)
Pantalla	No existe en el diseño
Rutas	<code>/perfil</code>

Estimación	4–6 h
Origen	RF9

Objetivo

Permitir que el usuario actualice sus datos. Cubre el requisito funcional RF9 de la tesis, que compromete esta funcionalidad ante el tribunal.

Incluye

- Endpoint 23: actualizar `fullName`; el docente además su email
- Endpoint 24: cambio de contraseña del docente
- Pantalla de perfil con datos de solo lectura y campos editables
- Acceso desde la cabecera del shell
- Confirmación visual al guardar

No incluye

- Que el alumno modifique `studentCode` o `classCode` — prohibido
- Foto de perfil
- Recuperación de contraseña

Criterios de aceptación

- ☐ Un alumno puede cambiar su nombre y el cambio se refleja en la cabecera y el ranking
- ☐ Un alumno **no puede** modificar `studentCode` ni `classCode`: se muestran de solo lectura y el servidor ignora el intento
- ☐ Un alumno que intenta cambiar `email` recibe `FORBIDDEN_ROLE`
- ☐ El docente puede cambiar su email; un email ya usado devuelve `EMAIL_ALREADY_REGISTERED`
- ☐ El docente cambia su contraseña dando la actual y una nueva
- ☐ Contraseña actual incorrecta devuelve `INCORRECT_CURRENT_PASSWORD`
- ☐ Una contraseña nueva de menos de 8 caracteres se rechaza
- ☐ La sesión sigue activa después de cambiar la contraseña
- ☐ El cambio se ve reflejado sin recargar
- ☐ Accesible en tres clics o menos
- ☐ Funciona en 375 px

Dependencias

F01, F02.

Riesgos

No hay mockup. Hay que diseñarla respetando el sistema del Documento de Diseño: fondo `bg`, tarjeta `surface`, bordes finos, sin sombras. Un formulario simple dentro de una `Card` alcanza.

`studentCode` y `classCode` son inmutables por diseño. Son la clave de cruce con los resultados de Google Forms. Si un alumno los cambia a mitad del piloto, sus datos pre/post dejan de conciliarse y se

pierde ese caso de la muestra. Mostralos, pero deshabilitados y con una nota.

Cambiar el nombre afecta al ranking. Es aceptable, pero conviene mantener la misma normalización del login para que el alumno pueda seguir entrando.

El token conserva el nombre viejo. El JWT lleva `fullName` en sus claims. Tras cambiarlo, el token sigue teniendo el anterior hasta que expire. La UI debe usar la respuesta de la API, no el claim, para mostrar el nombre.

F10 — Buscador de contenido

Dueño	Persona B
Endpoints	25
Colecciones	<code>glossaryTerms</code> (L), <code>modules</code> (L)
Pantalla	No existe en el diseño
Rutas	<code>/buscar</code> , <code>/glosario</code>
Estimación	6–9 h
Origen	RF8

Objetivo

Buscador que permite localizar módulos y términos de economía personal. Cubre el requisito funcional RF8 de la tesis.

Incluye

- Endpoint 25 con búsqueda de texto sobre glosario y módulos
- Colección `glossaryTerms` con 30 a 45 términos (Guía de BD, Bloque 5-B)
- Campo de búsqueda accesible desde el shell
- Resultados agrupados por tipo
- Vista de glosario completo
- Marcado de módulos no desbloqueados
- Estado vacío con sugerencia

No incluye

- Búsqueda dentro de las preguntas — **prohibido** (D-14)
- Historial de búsquedas
- Autocompletado

Criterios de aceptación

- ☐ Buscar un término del glosario lo devuelve como primer resultado

- ☐ Buscar el nombre de un módulo lo devuelve
- ☐ **Ningún resultado proviene de `questions`** — verificado leyendo la respuesta
- ☐ La búsqueda ignora mayúsculas y tildes: `interes` encuentra `interés`
- ☐ Menos de 2 caracteres devuelve `SEARCH_QUERY_TOO_SHORT`
- ☐ Sin coincidencias devuelve `200` con lista vacía, no 404
- ☐ Los módulos bloqueados aparecen atenuados y sin enlace
- ☐ El glosario completo se puede recorrer sin buscar
- ☐ Los resultados están ordenados por relevancia, no arbitrariamente
- ☐ Accesible desde cualquier pantalla del alumno
- ☐ Funciona en 375 px

Dependencias

F01, F02. Colección `glossaryTerms` sembrada.

Riesgos

Indexar `questions` invalidaría el piloto. Un alumno podría buscar palabras clave y encontrar preguntas antes de responderlas, en algunos casos infiriendo la respuesta. Es una restricción de integridad de la medición, no de producto (D-14).

El manejo de tildes es el punto débil. El *stemmer* en español de MongoDB no siempre iguala `interes` con `interés`, y escribir sin tildes es el comportamiento más común en un celular. La mitigación está en la guía de BD: incluir la versión sin tildes en `keywords`.

Sin \$meta: `'textScore'` los resultados salen desordenados y el buscador se siente roto aunque funcione.

El contenido es la mitad del trabajo. Los 30 a 45 términos son redacción pedagógica, no código. Empezalos temprano — es lo que suele atrasar esta feature.

No hay mockup. Diseñar respetando el sistema visual.

F11 — Contenido teórico de lecciones

Dueño	Persona B — la pantalla de teoría es la antesala directa de F04 (Motor de Lección), que ya es suya
Endpoints	27
Colecciones	<code>modules</code> (L, campo <code>theoryContent</code>)
Pantalla	Sin mockup en el Documento de Diseño — se diseña respetando el sistema visual
Rutas	<code>/leccion/:moduleId/teoria</code>
Estimación	10–14 h de código + redacción de 6 bloques teóricos
Origen	Tesis §4.6.3, resuelta por D-16

Resolución

La tesis §4.6.3 describe módulos con "lecciones cortas combinadas con actividades de práctica". Se decidió implementar esa parte teórica: **se prioriza el compromiso asumido ante el tribunal.**

Objetivo

Antes de responder las situaciones de un módulo, el alumno lee un bloque breve de contenido teórico. No modifica el motor de preguntas (F04): se inserta **antes**.

Mapa → Contenido teórico (F11, nueva) → Preguntas (F04) → Resultado (F05)

Incluye

- Endpoint 27, `GET /modules/:moduleId/content`
- Campo `theoryContent` (Markdown) y `estimatedReadingMinutes` en `modules`
- Pantalla nueva `/leccion/:moduleId/teoria`, con el contenido renderizado y un botón "Empezar preguntas" que dispara `POST /modules/:moduleId/attempts` (endpoint 7, F04) y navega a `/leccion/:attemptId`
- Redacción de los 6 bloques teóricos — repartidos entre ambos, igual que las preguntas y el glosario
- Botón "Jugar" del Mapa (F03) redirige ahora a `/leccion/:moduleId/teoria` en lugar de crear el intento directamente

No incluye

- El motor de preguntas en sí — F04, sin cambios
- Evaluación del contenido teórico — no hay preguntas sobre la teoría en sí, solo sobre las situaciones prácticas

Criterios de aceptación

- ☐ Al tocar "Jugar" en el Mapa, el alumno llega primero a la pantalla de teoría, no directo a las preguntas
- ☐ El contenido se renderiza como Markdown, con un componente sanitizado — nunca HTML crudo
- ☐ Un módulo bloqueado devuelve `MODULE_LOCKED` si se intenta acceder a su contenido por URL directa
- ☐ El botón "Empezar preguntas" crea el intento y navega correctamente
- ☐ Los 6 bloques teóricos están redactados y cargados por seed
- ☐ Funciona en 375 px

Dependencias

F01, F02, F03 (el botón "Jugar" cambia de destino). F04 no se modifica, solo se le antepone un paso.

Riesgos

Contenido, no código, es lo que atrasa esta feature. Redactar 6 bloques de teoría clara, breve y pedagógicamente sólida es más trabajo que el endpoint. Empezarlo temprano.

No convertir esto en una segunda evaluación. El bloque teórico es de lectura, sin preguntas ni bloqueo por tiempo. Si se le agrega fricción, se pierde el propósito y se parece más a F04 de lo necesario.

Longitud. Máximo 4000 caracteres por módulo — a propósito, corto. Es una lección breve, no un capítulo de libro de texto.

F12 — Integración Google Forms

Dueño	Persona B
Endpoints	21, 22
Colecciones	<code>testResults</code> (L/E), <code>users</code> (L)
Pantalla	Dentro del panel del docente
Estimación	5–8 h
Origen	D-02
Estado	 Opcional — Fase 2

Objetivo

Automatizar la carga de los resultados del pre-test y post-test desde Google Forms hacia la base, y mostrarlos en el panel del docente.

Es opcional por diseño. Los tests se aplican por Google Forms desde el inicio (D-01). Si no se implementa, los datos se exportan de Sheets a CSV y el análisis del Capítulo 5 se hace igual. No bloquea nada.

Incluye

- Endpoint 21: webhook autenticado por clave de integración
- Endpoint 22: resultados pre/post con cálculo de variación
- Script de Apps Script en la planilla de respuestas
- Conciliación por `studentCode`
- Sección en el panel del docente
- Columnas pre/post en la exportación CSV

No incluye

- Aplicar los tests dentro de la plataforma — descartado (D-01)
- Análisis estadístico formal — va en el Capítulo 5

Criterios de aceptación

- ☐ Enviar el formulario crea el registro en la base automáticamente
- ☐ Sin la clave correcta devuelve `INVALID_INTEGRATION_KEY`
- ☐ Un `studentCode` inexistente guarda igual con `matched: false` — el dato no se pierde
- ☐ Reenviar el mismo resultado no duplica: es idempotente
- ☐ `scorePercentage` se calcula en el servidor
- ☐ El panel muestra pre, post y variación por alumno
- ☐ El resumen calcula la mejora promedio
- ☐ El CSV incluye las columnas pre/post
- ☐ Alumnos con un solo test se muestran sin romper el cálculo

Dependencias

F08 para la visualización. Formularios creados y aplicados.

Riesgos

Nunca descartar un resultado sin coincidencia. Si un alumno tipea mal su código, el registro se guarda igual con `matched: false` y se concilia después. Perder una respuesta del piloto con una muestra de 10 a 20 estudiantes es un costo muy alto.

La clave de integración no va en el script a la vista. Usar las Script Properties de Apps Script.

El formulario debe pedir el código de alumno, con el mismo formato que la plataforma. Sin eso no hay cruce posible, y es un error que solo se descubre cuando ya se aplicó el test.

No es prioritaria. Si el cronograma aprieta, se descarta sin consecuencias.

Revisión de D-05. La decisión original decía: "Los alumnos se autentican con `fullName` + `studentCode` + `classCode`, sin contraseña. Coherente con el Documento de Diseño." Se reemplaza por lo siguiente:

#	Decisión	Fundamento
D-05 (revisada)	Los alumnos se autentican con <code>fullName</code> + <code>studentCode</code> + <code>classCode</code> + un PIN de 4 dígitos que ellos mismos eligen en su primer ingreso. Ya no hay acceso sin secreto. Si lo olvidan, el docente genera uno nuevo (endpoint 26).	El acceso sin ningún secreto resultaba insuficiente: dos alumnos que conocen el nombre y el código de un compañero —plausible en un curso de secundaria, con códigos correlativos— podían autenticarse como él. El PIN agrega un factor mínimo sin la fricción de una contraseña completa, apropiado para el contexto de aula y la edad de los usuarios.
D-15	El panel del docente usa Recharts para los gráficos (F08).	Librería de gráficos para React, liviana y con buena integración con Tailwind. Se le pasan los tokens de color de la §0.4 explícitamente — Recharts trae una paleta propia por defecto que rompería el sistema visual.
D-16	Se implementa el contenido teórico de las lecciones (F11). Cada módulo	Resuelve el conflicto entre la tesis §4.6.3 (que describe "lecciones cortas combinadas con

#	Decisión	Fundamento
	incluye un bloque de teoría breve, previo a las preguntas (endpoint 27).	actividades de práctica") y el Documento de Diseño (que solo contemplaba el motor de preguntas). Se prioriza el compromiso asumido ante el tribunal. Flujo: Mapa → Contenido teórico → Preguntas → Resultado.

Anexo — Historial de versiones

Versión	Fecha	Cambios
2.0	Agosto 2026	Reemplaza a ECOLINGO_CONTRATO_TECNICO.md v1.1, ECOLINGO_GUIA_BASE_DE_DATOS.md v1.1 y 02-FEATURES.md v1.0. Consolida los tres en un solo documento organizado por feature (API → BD → Frontend). PIN de 4 dígitos reemplaza el acceso sin contraseña de los alumnos (D-05 revisada). F11 (contenido teórico) se resuelve como implementada (D-16, endpoint 27). Se registra formalmente D-15 (Recharts).